Utilisez des architectures vérifiées

BeeGFS on NetApp with E-Series Storage

NetApp
March 21, 2024

This PDF was generated from https://docs.netapp.com/fr-fr/beegfs/beegfs-solution-overview.html on March 21, 2024. Always check docs.netapp.com for the latest.

Sommaire

Utilisez des architectures vérifiées 1
Présentation et configuration requise 1
Examen du design de la solution 10
Déploiement de la solution 24

Utilisez des architectures vérifiées

Présentation et configuration requise

Présentation de la solution

La solution BeeGFS sur NetApp associe le système de fichiers parallèle BeeGFS aux systèmes de stockage NetApp EF600 à une infrastructure fiable, évolutive et économique qui s'adapte aux besoins des workloads les plus exigeants.

Cette conception tire parti de la densité des performances fournie par les débits réseau et matériel de stockage et de serveur d'entreprise les plus récents, Ils requièrent des nœuds de fichiers dotés de deux processeurs AMD EPYC 7003 « Milan », ainsi que la prise en charge de PCIe 4.0 avec des connexions directes via une InfiniBand de 200 Go (HDR) pour des nœuds de bloc offrant une connectivité NVMe de bout en bout et NVMeOF via le protocole NVMe/IB.

Programme NVA

La solution BeeGFS sur NetApp fait partie du programme NVA (NetApp Verified Architecture), qui fournit aux clients des configurations de référence et des conseils de dimensionnement pour des workloads et des cas d'utilisation spécifiques. Les solutions NVA sont minutieusement testées et conçues pour réduire les risques de déploiement et accélérer le délai de mise sur le marché.

Cas d'utilisation

Les utilisations suivantes s'appliquent à la solution BeeGFS sur NetApp :

- L'intelligence artificielle (IA), comprenant le machine learning (ML), le deep learning (DL), le traitement du langage naturel à grande échelle (NLP) et la compréhension du langage naturel (NLU). Pour plus d'informations, voir "BeeGFS pour l'IA : faits plutôt que fiction".
- Informatique hautes performances (HPC), y compris les applications accélérées par MPI (interface de transmission de messages) et d'autres techniques informatiques distribuées. Pour plus d'informations, voir "Pourquoi BeeGFS va bien au-delà de l'HPC".
- Charges de travail applicatives caractérisées par :
  - Lecture ou écriture dans des fichiers supérieurs à 1 Go
  - Lecture ou écriture dans le même fichier par plusieurs clients (dizaines, centaines et milliers)
- Jeux de données de plusieurs téraoctets ou de plusieurs pétaoctets.
- Environnements qui nécessitent un seul espace de noms de stockage optimal pour un mélange de fichiers de petite ou de grande taille.

Avantages

Voici les principaux avantages de BeeGFS sur NetApp :

- La disponibilité de conceptions matérielles vérifiées permet l'intégration complète des composants matériels et logiciels pour assurer des performances prévisibles et une fiabilité optimale.
- Déploiement et gestion avec Ansible pour une simplicité et une cohérence à grande échelle.
- Contrôle et observabilité fournis avec l'analyseur de performance E-Series et le plug-in BeeGFS. Pour plus

d'informations, voir "Présentation d'un cadre de surveillance des solutions NetApp E-Series".

- Haute disponibilité dotée d'une architecture de disques partagés qui assure la durabilité et la disponibilité des données.
- Prise en charge des fonctionnalités modernes de gestion et d'orchestration des workloads à l'aide de conteneurs et de Kubernetes. Pour plus d'informations, voir "Kubernetes et BeeGFS : un récit d'investissement pérenne".

Architecture HAUTE DISPONIBILITÉ

BeeGFS sur NetApp étend les fonctionnalités de la version BeeGFS Enterprise en créant une solution entièrement intégrée avec du matériel NetApp qui offre une architecture haute disponibilité (HA) de disque partagé.

> L'édition communautaire BeeGFS peut être utilisée gratuitement. Cependant, l'édition entreprise exige l'achat d'un contrat d'abonnement de support professionnel auprès d'un partenaire comme NetApp. L'édition entreprise permet d'utiliser plusieurs fonctions supplémentaires, notamment la résilience, l'application de quotas et les pools de stockage.

La figure suivante compare les architectures haute disponibilité sans partage et à disque partagé.

[]

Pour plus d'informations, voir "Annonce de la haute disponibilité pour BeeGFS prise en charge par NetApp".

Ansible

BeeGFS sur NetApp est fourni et déployé à l'aide d'Ansible Automation, qui est hébergé sur GitHub et Ansible Galaxy (la collection BeeGFS est disponible sur "Galaxy Ansible" et "NetApp E-Series GitHub"). Bien qu'Ansible soit principalement testé avec le matériel utilisé pour assembler les éléments de base BeeGFS, vous pouvez le configurer de sorte qu'il s'exécute sur presque tous les serveurs x86 à l'aide d'une distribution Linux prise en charge.

Pour plus d'informations, voir "Déploiement de BeeGFS avec E-Series Storage".

Générations de conception

La solution BeeGFS sur NetApp est actuellement de sa deuxième conception générationnelle.

La première et la deuxième génération incluent une architecture de base qui inclut un système de fichiers BeeGFS et un système de stockage NVMe EF600. Cependant, la deuxième génération s'appuie sur la première pour inclure ces avantages supplémentaires :

- Doublez les performances et la capacité, tout en n'ajoutant que 2U d'espace rack
- Haute disponibilité (HA) qui repose sur un design matériel à deux niveaux à disque partagé
- Qualification externe pour les architectures NVIDIA DGX A100 SuperPOD et NVIDIA BasePOD

Seconde conception générationnelle

La deuxième génération de BeeGFS sur NetApp est optimisée pour répondre aux besoins de performance des workloads exigeants, notamment l'informatique haute performance (HPC) et le machine learning de style HPC (ML), le deep learning (DL) et d'intelligence artificielle (IA) similaires. Grâce à l'architecture haute disponibilité

(HA) à disque partagé, la solution BeeGFS sur NetApp répond également aux besoins de durabilité et de disponibilité des données des entreprises et autres entreprises qui ne peuvent pas se permettre de temps d'indisponibilité ni de perte de données quand elles recherchent un stockage qui peut évoluer pour rester en phase avec leurs charges de travail et leurs cas d'utilisation. Non seulement NetApp vérifie cette solution, mais elle a également réussi la qualification externe en tant qu'option de stockage pour le NVIDIA DGX SuperPOD et le DGX BasePOD.

Première conception générationnelle

La première génération de BeeGFS sur NetApp a été conçue pour les workloads d'apprentissage machine (ML) et d'intelligence artificielle (IA) à l'aide des systèmes de stockage NVMe NetApp EF600, du système de fichiers parallèle BeeGFS, des systèmes NVIDIA DGX™ A100 et des switchs IB NVIDIA® Mellanox® Quantum™ QM8700 200 Gbit/s. Cette conception comprend également une InfiniBand (IB) 200 Gbit/s pour la structure d'interconnexion des clusters de stockage et de calcul, afin d'offrir une architecture entièrement basée sur IB pour les charges de travail hautes performances.

Pour plus d'informations sur la première génération, voir "NetApp EF-Series ai avec NVIDIA DGX A100 et BeeGFS".

Présentation de l'architecture

La solution BeeGFS sur NetApp inclut des critères de conception architecturale qui permettent de déterminer l'équipement, le câblage et les configurations qui sont requis pour prendre en charge les workloads validés.

Architecture modulaire

Le système de fichiers BeeGFS peut être déployé et adapté de différentes manières, en fonction des besoins en stockage. Par exemple, certains cas d'utilisation mettant en avant de nombreux fichiers de petite taille bénéficieront d'une performance et d'une capacité supplémentaires de métadonnées, tandis que les cas d'utilisation comportant moins de fichiers volumineux peuvent favoriser une capacité de stockage et des performances supérieures pour le contenu réel des fichiers. Ces considérations ont un impact sur les différentes dimensions du déploiement d'un système de fichiers parallèle, ce qui ajoute de la complexité à la conception et au déploiement d'un système de fichiers.

En réponse à ces défis, NetApp a conçu une architecture d'éléments de base standard qui permet une évolutivité horizontale de chaque catégorie. De façon générale, les éléments de base BeeGFS sont déployés dans l'un des trois profils de configuration suivants :

- Un élément de base unique, incluant la gestion BeeGFS, les métadonnées et les services de stockage
- Des métadonnées BeeGFS plus un élément de base du stockage
- Un élément de base de stockage BeeGFS uniquement

Le seul changement matériel entre ces trois options est l'utilisation de lecteurs plus petits pour les métadonnées BeeGFS. Dans le cas contraire, toutes les modifications de configuration sont appliquées via le logiciel. En outre, avec Ansible comme moteur de déploiement, la configuration du profil souhaité pour un élément de base particulier simplifie les tâches de configuration.

Pour plus de détails, voir Conception matérielle vérifiée.

Services de système de fichiers

Le système de fichiers BeeGFS inclut les principaux services suivants :

- **Service de gestion.** registres et contrôle tous les autres services.
- **Service de stockage.** stocke le contenu des fichiers d'utilisateur distribués appelé fichiers de bloc de données.
- **Service de métadonnées.** assure le suivi de la disposition du système de fichiers, du répertoire, des attributs de fichier, etc.
- **Service client.** monte le système de fichiers pour accéder aux données stockées.

La figure suivante présente les composants et les relations de la solution BeeGFS utilisés avec les systèmes NetApp E-Series.

[]

En tant que système de fichiers parallèle, BeeGFS répartit ses fichiers sur plusieurs nœuds de serveur afin de maximiser les performances en lecture/écriture et l'évolutivité. Les nœuds de serveur fonctionnent ensemble pour fournir un système de fichiers unique pouvant être monté et accessible simultanément par d'autres nœuds de serveur, communément appelés *clients*. Ces clients peuvent voir et consommer le système de fichiers distribué de la même manière qu'un système de fichiers local tel que NTFS, XFS ou ext4.

Les quatre services principaux fonctionnent sur un large éventail de distributions Linux prises en charge et communiquent via n'importe quel réseau compatible TCP/IP ou RDMA, y compris InfiniBand (IB), Omni-Path (OPA) et RDMA over Converged Ethernet (RoCE). Les services de serveur BeeGFS (gestion, stockage et métadonnées) sont des démons d'espace utilisateur, alors que le client est un module de noyau natif (sans patchless). Tous les composants peuvent être installés ou mis à jour sans redémarrage. Vous pouvez en outre exécuter n'importe quelle combinaison de services sur le même nœud.

Nœuds vérifiés

La solution BeeGFS sur NetApp inclut les nœuds vérifiés suivants : le système de stockage NetApp EF600 (nœud bloc) et le serveur Lenovo ThinkSystem SR665 (nœud de fichier).

Nœud bloc : système de stockage EF600

La baie 100 % Flash NetApp EF600 assure un accès aux données cohérent en temps quasi réel avec un nombre illimité de workloads. Pour alimenter rapidement et en continu les données des applications d'IA et HPC, les systèmes de stockage EF600 fournissent jusqu'à 2 millions d'IOPS en lecture cache, des temps de réponse inférieurs à 100 microsecondes et un débit de lecture séquentielle de 42 Gbit/s dans un seul boîtier.

Nœud de fichier : serveur Lenovo ThinkSystem SR665

Le SR665 est un serveur 2U à deux sockets doté de la technologie PCIe 4.0. Lorsqu'elle est configurée pour répondre aux besoins de cette solution, elle permet d'exécuter des services de fichiers BeeGFS dans une configuration bien équilibrée avec la disponibilité du débit et des IOPS fournie par les nœuds E-Series à connexion directe.

Pour plus d'informations sur le Lenovo SR665, reportez-vous à la section "Site Web de Lenovo".

Conception matérielle vérifiée

Les éléments de base de la solution (illustrée dans la figure ci-dessous) utilisent deux serveurs PCIe 4.0 à double socket pour la couche de fichiers BeeGFS et deux systèmes de stockage EF600 comme la couche bloc.

[]

> Chaque élément de base inclut deux nœuds de fichiers BeeGFS, un minimum de deux éléments de base est requis pour établir le quorum dans le cluster de basculement. Vous pouvez configurer un cluster à deux nœuds, mais cette configuration présente des limites susceptibles d'empêcher un basculement réussi. Si vous avez besoin d'un cluster à deux nœuds, vous pouvez incorporer un troisième périphérique en tant qu'Tiebreaker (mais cette conception n'est pas couverte par ce site).

Chaque élément de base offre une haute disponibilité via une conception matérielle à deux niveaux qui sépare les domaines de défaillance des couches de fichiers et de blocs. Chaque Tier peut basculer indépendamment, ce qui améliore la résilience et réduit le risque de pannes en cascade. L'utilisation de la technologie HDR InfiniBand associée à NVMeOF offre un débit élevé et une latence minimale entre les nœuds de fichiers et de blocs, avec une redondance complète et une sursouscription suffisante des liaisons pour éviter que la conception désagrégée ne devienne un goulot d'étranglement, même si le système est partiellement dégradé.

La solution BeeGFS sur NetApp s'exécute sur tous les éléments de base du déploiement. Le premier élément de base déployé doit exécuter des services de gestion BeeGFS, de métadonnées et de stockage (appelé élément de base). Tous les éléments de base ultérieurs sont configurés via le logiciel pour exécuter des services de métadonnées et de stockage BeeGFS ou uniquement des services de stockage. La disponibilité de différents profils de configuration pour chaque élément de base permet de faire évoluer les métadonnées du système de fichiers ou la capacité et les performances du stockage en utilisant les mêmes plateformes matérielles sous-jacentes et la même conception d'éléments de base.

Jusqu'à cinq éléments de base sont combinés dans un cluster Linux HA autonome, garantissant un nombre raisonnable de ressources par gestionnaire de ressources de cluster (Pacemaker) et réduisant la surcharge de messagerie requise pour maintenir la synchronisation des membres du cluster (Corosync). Un minimum de deux blocs de construction par grappe est recommandé pour permettre à un nombre suffisant de membres d'établir le quorum. Un ou plusieurs de ces clusters autonomes BeeGFS HA sont combinés pour créer un système de fichiers BeeGFS (illustré dans la figure suivante) qui est accessible aux clients comme un seul espace de noms de stockage.

[]

Bien qu'au final, le nombre d'éléments de base par rack dépend des besoins en alimentation et en climatisation d'un site donné, La solution a été conçue de sorte à déployer jusqu'à cinq éléments de base dans un seul rack 42U tout en laissant de l'espace à deux commutateurs InfiniBand 1U utilisés pour le réseau de stockage/données. Chaque module nécessite huit ports IB (quatre par commutateur pour la redondance). Ainsi, cinq blocs de construction laissent la moitié des ports sur un commutateur InfiniBand HDR à 40 ports (comme le NVIDIA QM8700) disponible pour implémenter une topologie FAT-Tree ou non bloquante similaire. Cette configuration garantit une évolutivité verticale du nombre de racks de stockage ou de calcul/GPU sans goulets d'étranglement du réseau. Il est également possible d'utiliser une structure de stockage sursouscrite à l'aide des recommandations du fournisseur de la structure de stockage.

L'image suivante montre une topologie arborescence FAT à 80 nœuds.

[]

En utilisant Ansible comme moteur de déploiement pour déployer BeeGFS sur NetApp, les administrateurs peuvent gérer tout l'environnement en utilisant une infrastructure moderne comme du code. Cela simplifie considérablement le fonctionnement d'un système complexe, permettant aux administrateurs de définir et d'ajuster la configuration en un seul emplacement, puis de s'assurer de son application cohérente quelle que soit la taille de l'environnement. La collection BeeGFS est disponible à partir de "Galaxy Ansible" et "NetApp E-Series GitHub".

Exigences techniques

Pour implémenter la solution BeeGFS sur NetApp, assurez-vous que votre environnement répond aux exigences technologiques.

Configuration matérielle requise

Le tableau suivant répertorie les composants matériels requis pour implémenter un élément de base de deuxième génération unique de la solution BeeGFS sur NetApp.

Ils peuvent varier selon la mise en œuvre de la solution et les besoins du client.

Nombre	Composant matériel	De formation
2	Nœuds de fichiers BeeGFS.	Pour atteindre les performances attendues, chaque nœud de fichier doit remplir ou dépasser la configuration suivante. **Processeurs:** • 2 AMD EPYC 7343 16C 3.2 GHz. • Configuré comme deux zones NUMA. **Mémoire:** • 256 GO. • 16x 16 Go TruDDR4 3800 MHz (2Rx8 1,2V) RDIMM-A (préférez des barrettes DIMM plus petites sur moins de barrettes DIMM plus grandes). • Rempli pour maximiser la bande passante de la mémoire. **Extension PCIe : quatre emplacements PCE Gen4 x16 :** • Deux emplacements par zone NUMA. • Chaque slot doit fournir suffisamment d'alimentation/de refroidissement pour l'adaptateur Mellanox MCX4653106A-HDAT. **Divers:** • Deux disques SATA 1 To 7200 tr/min (ou comparables) configurés en RAID 1 pour le système d'exploitation. • Adaptateur 10GbE OCP 3.0 (ou équivalent) pour la gestion du système d'exploitation intrabande. • 1GbE BMC avec Redfish API pour la gestion de serveurs hors bande. • Deux blocs d'alimentation et ventilateurs de performances remplaçables à chaud. • Les câbles InfiniBand optiques Mellanox doivent être pris en charge si nécessaire pour atteindre les commutateurs InfiniBand de stockage. **Lenovo SR665:** • Un modèle NetApp personnalisé inclut la version requise du firmware du contrôleur XClarity requise pour prendre en charge les adaptateurs Mellanox ConnectX-6 à deux ports. Contactez NetApp pour en savoir plus sur le processus de commande.
8	Mellanox ConnectX-6 HCA (pour les nœuds de fichiers).	• Adaptateurs de canal hôte MCX653106A-HDAT (HDR IB 200 Go, QSFP56 à deux ports, PCIe4.0 x16).

Nombre	Composant matériel	De formation
8	1 m de câbles InfiniBand HDR (pour les connexions directes de nœud/bloc).	• MCP1650-H001E30 (câble en cuivre passif Mellanox de 1 m, IB HDR, jusqu'à 200 Gbit/s, QSFP56, 30 AWG). La longueur peut être ajustée pour tenir compte des distances plus longues entre les nœuds de fichier et de bloc si nécessaire.
8	Câbles InfiniBand HDR (pour les connexions de nœud de fichiers/commutateur de stockage)	Nécessite des câbles HDR InfiniBand (émetteurs-récepteurs QSFP56) de la longueur appropriée pour connecter des nœuds de fichiers aux switchs de lames de stockage. Les options possibles sont les suivantes : • MCP1650-H002E26 (câble en cuivre passif Mellanox de 2 m, IB HDR, jusqu'à 200 Go/s, QSFP56, 30 AWG). • MFS1S00-H003E (câble à fibre active Mellanox de 3 m, IB HDR, jusqu'à 200 Go/s, QSFP56).
2	Nœuds de blocs E-Series	Deux contrôleurs EF600 configurés comme suit : • Mémoire : 256 Go (128 Go par contrôleur). • Adaptateur : 2 ports 200 Go/HDR (NVMe/IB). • Disques : configurés pour correspondre à la capacité souhaitée.

Configuration logicielle requise

Pour des performances et une fiabilité prévisibles, les versions de la solution BeeGFS sur NetApp sont testées avec des versions spécifiques des composants logiciels requis pour implémenter la solution.

Configuration requise pour le déploiement de logiciels

Le tableau suivant répertorie les exigences logicielles déployées automatiquement dans le cadre du déploiement BeeGFS basé sur Ansible.

Logiciel	Version
BeeGFS	7.2.6
Corosync	3.1.5-1
Stimulateur cardiaque	2.1.0-8
OpenSM	openmm-5.9.0 (à partir de mlnx_ofed 5.4-1.0.3.0) Uniquement nécessaire pour les connexions directes afin d'activer la virtualisation.

Configuration requise pour le nœud de contrôle Ansible

La solution BeeGFS sur NetApp est déployée et gérée à partir d'un nœud de contrôle Ansible. Pour plus

d'informations, reportez-vous à la section "Documentation Ansible".

Les exigences logicielles répertoriées dans les tableaux suivants sont spécifiques à la version de la collection NetApp BeeGFS Ansible indiquée ci-dessous.

Logiciel	Version
Ansible	2.11 lorsqu'il est installé via pip : ansible-4.7.0 et ansible-core < 2.12,>=2.11.6
Python	3.9
Packs Python supplémentaires	Cryptographie-35.0.0, netaddr-0.8.0
Collection BeeGFS Ansible	3.0.0

Configuration requise pour les nœuds de fichiers

Logiciel	Version
Red Hat Enterprise Linux	Red Hat 8.4 Server Physical avec haute disponibilité (2 sockets). Les nœuds de fichiers nécessitent un abonnement Red Hat Enterprise Linux Server valide et le module complémentaire haute disponibilité de Red Hat Enterprise Linux.
Noyau Linux	4.18.0-305.25.1.el8_4.x86_64
Pilotes InfiniBand / RDMA	Boîte de réception
Microprogramme HCA ConnectX-6	FW : 20.31.1014
PXE : 3.6.0403	UEFI : 14.24.0013

Exigences liées aux nœuds en mode bloc EF600

Logiciel	Version
SANtricity OS	11.70.2
NVSRAM	N6000-872834-D06.dlp
Micrologiciel de lecteur	Dernière version disponible pour les modèles de lecteurs utilisés.

Supplémentaires requise

L'équipement indiqué dans le tableau suivant a été utilisé pour la validation, mais des alternatives appropriées peuvent être utilisées si nécessaire. En général, NetApp recommande d'exécuter les dernières versions de logiciels pour éviter les problèmes inattendus.

Composant matériel	Logiciel installé
• 2 switchs InfiniBand 200 Gb Mellanox MQ700	• Micrologiciel 3.9.2110
1x nœud de contrôle Ansible (virtualisé) : • Processeurs : processeur Intel® Xeon® Gold 6146 à 3,20 GHz • Mémoire: 8 Go • Stockage local : 24 Go	• CentOS Linux 8.4.2105 • Noyau 4.18.0-305.3.1.el8.x86_64 Les versions Ansible et Python installés correspondent à celles du tableau ci-dessus.
10x clients BeeGFS (nœuds CPU): • Processeur : 1 processeur AMD EPYC 7302 16 cœurs à 3 GHz • Mémoire : 128 Go • Réseau : 2 x Mellanox MCX4653106A-HDAT (un port connecté par adaptateur).	• Ubuntu 20.04 • Noyau: 5.4.0-100-générique • Pilotes InfiniBand : Mellanox OFED 5.4-1.0.3.0
1x BeeGFS client (nœud GPU): • Processeurs : 2 processeurs AMD EPYC 7742 64 cœurs à 2,25 GHz • Mémoire : 1 To • Réseau : 2 x Mellanox MCX4653106A-HDAT (un port connecté par adaptateur). Ce système est basé sur la plateforme NVIDIA IDA HGX A100 et inclut quatre GPU A100.	• Ubuntu 20.04 • Noyau: 5.4.0-100-générique • Pilotes InfiniBand : Mellanox OFED 5.4-1.0.3.0

Examen du design de la solution

Présentation du design

Pour prendre en charge la solution BeeGFS sur NetApp, qui associe le système de fichiers parallèle BeeGFS et les systèmes de stockage NetApp EF600, vous devez utiliser un équipement, un câblage et des configurations spécifiques.

En savoir plus :

- "Configuration matérielle"
- "Configuration logicielle"
- "Vérification de la conception"
- "Instructions de dimensionnement"
- "Réglage des performances"

Architectures dérivées avec des variations de conception et de performances :

- "Élément de base haute capacité"

Configuration matérielle

La configuration matérielle de BeeGFS sur NetApp inclut des nœuds de fichiers et le câblage réseau.

Configuration de nœud de fichiers

Les nœuds de fichiers ont deux sockets de CPU configurés en zones NUMA distinctes, qui incluent un accès local à un nombre égal de slots PCIe et de mémoire.

Les adaptateurs InfiniBand doivent être placés dans les connecteurs ou les cartes de montage PCI appropriés, de sorte que la charge de travail soit équilibrée sur les voies PCIe et les canaux de mémoire disponibles. Pour équilibrer la charge de travail, vous pouvez isoler intégralement le travail des services BeeGFS vers un nœud NUMA particulier. L'objectif est d'atteindre les mêmes performances pour chaque nœud de fichiers que s'il s'agissait de deux serveurs à socket unique indépendants.

La figure suivante montre la configuration NUMA du nœud de fichiers.

[]

Les processus BeeGFS sont épinglés à une zone NUMA particulière pour s'assurer que les interfaces utilisées se trouvent dans la même zone. Cette configuration évite d'avoir besoin d'un accès à distance via la connexion inter-socket. La connexion inter-sockets est parfois appelée liaison QPI ou GMI2 ; même dans les architectures de processeurs modernes, ils peuvent être un goulot d'étranglement lors de l'utilisation de réseaux haut débit comme HDR InfiniBand.

Configuration des câbles réseau

Dans un élément de base, chaque nœud de fichier est connecté à deux nœuds de bloc grâce à quatre connexions InfiniBand redondantes. En outre, chaque nœud de fichiers dispose de quatre connexions redondantes au réseau de stockage InfiniBand.

Dans la figure suivante, notez que :

- Tous les ports de nœuds de fichiers indiqués en vert sont utilisés pour la connexion au maillage Storage Fabric ; tous les autres ports de nœuds de fichiers sont les connexions directes aux nœuds de blocs.
- Deux ports InfiniBand d'une zone NUMA spécifique se connectent aux contrôleurs A et B du même nœud de bloc.
- Les ports du nœud NUMA 0 se connectent toujours au premier nœud de bloc.
- Les ports du nœud NUMA 1 se connectent au second nœud de bloc.

[]

> Pour les réseaux de stockage avec commutateurs redondants, les ports indiqués en vert clair doivent se connecter à un commutateur et les ports en vert foncé à un autre commutateur.

La configuration de câblage illustrée dans la figure permet à chaque service BeeGFS de :

- Exécuter dans la même zone NUMA, quel que soit le nœud de fichier qui exécute le service BeeGFS.
- Disposer de chemins optimaux secondaires au réseau de stockage frontal et aux nœuds de blocs internes, quel que soit l'endroit où une défaillance se produit
- Réduisez l'impact sur la performance si un nœud de fichiers ou un contrôleur d'un nœud de blocs nécessite une maintenance.

Câblage pour exploiter la bande passante

Pour exploiter la bande passante bidirectionnelle PCIe complète, vérifiez que un port de chaque adaptateur InfiniBand se connecte à la structure de stockage, et que l'autre port est connecté à un nœud de bloc. La vitesse maximale théorique d'un port InfiniBand HDR est de 25 Gbit/s (ne tenant pas compte de la signalisation et des autres frais généraux). La bande passante unidirectionnelle maximale d'un slot PCIe 4.0 x16 est de 32 Gbit/s, ce qui crée un goulot d'étranglement potentiel lors de l'implémentation de nœuds de fichiers intégrant des adaptateurs InfiniBand à double port qui peuvent théoriquement gérer 50 Gbit/s de bande passante.

La figure suivante montre la conception de câblage utilisée pour exploiter la bande passante bidirectionnelle PCIe complète.

[]

Pour chaque service BeeGFS, utilisez la même carte pour connecter le port préféré utilisé pour le trafic client avec le chemin vers le contrôleur de nœuds de bloc qui est le principal propriétaire de ces volumes de services. Pour plus d'informations, voir "Configuration logicielle".

Configuration logicielle

La configuration logicielle de BeeGFS sur NetApp inclut des composants réseau BeeGFS, des nœuds de bloc EF600, des nœuds de fichiers BeeGFS, des groupes de ressources et des services BeeGFS.

Configuration réseau BeeGFS

La configuration du réseau BeeGFS comprend les composants suivants.

- **IP flottantes** les adresses IP flottantes sont un type d'adresse IP virtuelle qui peut être routée dynamiquement vers n'importe quel serveur du même réseau. Plusieurs serveurs peuvent posséder la même adresse IP flottante, mais elle ne peut être active que sur un seul serveur à la fois.

  Chaque service de serveur BeeGFS possède sa propre adresse IP qui peut se déplacer entre les nœuds de fichiers en fonction de l'emplacement d'exécution du service de serveur BeeGFS. Cette configuration IP flottante permet à chaque service de basculer indépendamment vers l'autre nœud de fichiers. Le client a simplement besoin de connaître l'adresse IP d'un service BeeGFS particulier; il n'est pas nécessaire de savoir quel nœud de fichier exécute actuellement ce service.

- **Configuration multi-homing du serveur BeeGFS** pour augmenter la densité de la solution, chaque nœud de fichiers a plusieurs interfaces de stockage avec des adresses IP configurées dans le même sous-réseau IP.

  Des configurations supplémentaires sont nécessaires pour s'assurer que cette configuration fonctionne comme prévu avec la pile réseau Linux, car par défaut, les requêtes à une interface peuvent être traitées sur une autre interface si leurs adresses IP se trouvent dans le même sous-réseau. Outre d'autres inconvénients, ce comportement par défaut rend impossible l'établissement ou la maintenance des connexions RDMA.

Le déploiement Ansible gère le serrage du comportement de la RP (reverse path) et du protocole ARP (Address Resolution Protocol), ainsi que la vérification du démarrage et de l'arrêt d'adresses IP flottantes ; les routes et règles IP correspondantes sont créées dynamiquement pour permettre à la configuration réseau multihomée de fonctionner correctement.

- **Configuration multi-rail client BeeGFS** *Multi-rail* désigne la capacité d'une application d'utiliser des « rails » de réseau indépendants multiples pour augmenter les performances.

  Bien que BeeGFS peut utiliser RDMA pour la connectivité, BeeGFS utilise IPoIB pour simplifier la recherche et la création de connexions RDMA. Pour permettre aux clients BeeGFS d'utiliser plusieurs interfaces InfiniBand, vous pouvez configurer chaque client avec une adresse IP située dans un sous-réseau différent, puis configurer les interfaces préférées pour la moitié des services serveur BeeGFS dans chaque sous-réseau.

  Dans le schéma suivant, les interfaces surlignées en vert clair se trouvent dans un sous-réseau IP (par exemple, `100.127.0.0/16`) et les interfaces vert foncé sont situées dans un autre sous-réseau (par exemple, `100.128.0.0/16`).

  La figure suivante montre l'équilibrage du trafic sur plusieurs interfaces client BeeGFS.

  []

  Comme chaque fichier de BeeGFS est généralement réparti sur plusieurs services de stockage, la configuration multi-rail permet au client d'atteindre un débit supérieur à celui d'un seul port InfiniBand. Par exemple, l'exemple de code suivant montre une configuration commune de répartition des fichiers qui permet au client d'équilibrer le trafic entre les deux interfaces :

```
root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
Entry type: file
EntryID: 11D-624759A9-65
Metadata node: meta_01_tgt_0101 [ID: 101]
Stripe pattern details:
+ Type: RAID0
+ Chunksize: 1M
+ Number of storage targets: desired: 4; actual: 4
+ Storage targets:
  + 101 @ stor_01_tgt_0101 [ID: 101]
  + 102 @ stor_01_tgt_0101 [ID: 101]
  + 201 @ stor_02_tgt_0201 [ID: 201]
  + 202 @ stor_02_tgt_0201 [ID: 201]
```

  L'utilisation de deux sous-réseaux IPoIB est une distinction logique. Si vous le souhaitez, vous pouvez utiliser un seul sous-réseau InfiniBand physique (réseau de stockage).

> Le support multi-rail a été ajouté dans BeeGFS 7.3.0 pour permettre l'utilisation de plusieurs interfaces IB dans un seul sous-réseau IPoIB. Le design de la solution BeeGFS sur NetApp a été développé avant la disponibilité générale de BeeGFS 7.3.0, et démontre ainsi l'utilisation de deux sous-réseaux IP pour utiliser deux interfaces IB sur les clients BeeGFS. L'un des avantages de l'approche de sous-réseau IP multiple est d'éliminer la nécessité de configurer plusieurs systèmes d'accueil sur les nœuds clients BeeGFS (pour plus d'informations, voir "Prise en charge de RDMA BeeGFS").

Configuration de nœud en mode bloc EF600

Les nœuds de blocs comprennent deux contrôleurs RAID actifs/actifs avec accès partagé au même ensemble de lecteurs. En général, chaque contrôleur possède la moitié des volumes configurés sur le système, mais il peut reprendre l'autre contrôleur si nécessaire.

Le logiciel de chemins d'accès multiples des nœuds de fichiers détermine le chemin actif et optimisé vers chaque volume et le déplace automatiquement vers l'autre chemin d'accès en cas de défaillance du câble, de l'adaptateur ou du contrôleur.

Le schéma suivant illustre la disposition du contrôleur dans les nœuds de bloc EF600.

[]

Pour faciliter la solution haute disponibilité du disque partagé, les volumes sont mappés sur les deux nœuds de fichiers de manière à ce qu'ils puissent prendre en charge les uns les autres selon les besoins. Le diagramme suivant montre un exemple de configuration du service BeeGFS et de la propriété du volume préféré pour des performances maximales. L'interface à gauche de chaque service BeeGFS indique l'interface préférée que les clients et les autres services utilisent pour le contacter.

[]

Dans l'exemple précédent, les clients et les services serveur préfèrent communiquer avec le service de stockage 1 via l'interface i1b. Le service de stockage 1 utilise l'interface i1a comme chemin préféré pour communiquer avec ses volumes (Storage_tgt_101, 102) sur le contrôleur A du premier nœud de bloc. Cette configuration utilise la bande passante PCIe bidirectionnelle complète disponible pour l'adaptateur InfiniBand et offre de meilleures performances avec un adaptateur HDR InfiniBand à deux ports que le tout avec PCIe 4.0.

Configuration de nœud de fichier BeeGFS

Les nœuds de fichiers BeeGFS sont configurés dans un cluster haute disponibilité (HA) pour faciliter le basculement des services BeeGFS entre plusieurs nœuds de fichiers.

La conception du cluster HA repose sur deux projets Linux HA largement utilisés : Corosync pour l'appartenance à un cluster et Pacemaker pour la gestion des ressources de cluster. Pour plus d'informations, voir "Formation Red Hat pour les modules complémentaires haute disponibilité".

NetApp a rédigé et étendu plusieurs agents de ressources OCF (Open Cluster Framework) pour permettre au cluster de démarrer et de surveiller intelligemment les ressources BeeGFS.

Clusters HA BeeGFS

De façon générale, lorsque vous démarrez un service BeeGFS (avec ou sans HA), quelques ressources doivent être en place :

- Adresses IP où le service est accessible, généralement configurées par Network Manager.
- Les systèmes de fichiers sous-jacents sont utilisés comme cibles de BeeGFS pour stocker des données.

  Celles-ci sont généralement définies dans `/etc/fstab` Et monté par systemd.

- Un service systemd responsable du démarrage des processus BeeGFS lorsque les autres ressources sont prêtes.

  Sans logiciel supplémentaire, ces ressources ne démarrent que sur un seul nœud de fichiers. Par conséquent, si le nœud de fichier passe hors ligne, une partie du système de fichiers BeeGFS est inaccessible.

Comme plusieurs nœuds peuvent démarrer chaque service BeeGFS, Pacemaker doit s'assurer que chaque service et chaque ressource dépendante sont exécutés sur un seul nœud à la fois. Par exemple, si deux nœuds tentent de démarrer le même service BeeGFS, il y a un risque de corruption des données s'ils essaient tous les deux d'écrire sur les mêmes fichiers sur la cible sous-jacente. Pour éviter ce scénario, Pacemaker utilise Corosync pour maintenir en toute fiabilité l'état du cluster global en mode synchrone sur tous les nœuds et établir le quorum.

En cas de défaillance dans le cluster, Pacemaker réagit et redémarre les ressources BeeGFS sur un autre nœud. Dans certains cas, il se peut que Pacemaker ne puisse pas communiquer avec le nœud défectueux d'origine pour confirmer que les ressources sont arrêtées. Pour vérifier que le nœud est arrêté avant de redémarrer les ressources BeeGFS ailleurs, Pacemaker déligne le nœud défectueux, idéalement en retirant l'alimentation.

De nombreux agents d'escrime open source sont disponibles pour permettre à Pacemaker de verrouiller un nœud avec une unité de distribution d'alimentation (PDU) ou à l'aide du contrôleur BMC (Baseboard Management Controller) de serveur avec des API telles que Redfish.

Lorsque BeeGFS est exécuté dans un cluster HA, tous les services BeeGFS et les ressources sous-jacentes sont gérés par Pacemaker dans des groupes de ressources. Chaque service BeeGFS et les ressources dont il dépend sont configurés dans un groupe de ressources qui assure le démarrage et l'arrêt des ressources dans le bon ordre et qui sont situés sur le même nœud.

Pour chaque groupe de ressources BeeGFS, Pacemaker exécute une ressource de surveillance BeeGFS personnalisée qui est chargée de détecter les conditions de défaillance et de déclencher intelligemment les basculements lorsqu'un service BeeGFS n'est plus accessible sur un nœud particulier.

La figure suivante montre les services et les dépendances de BeeGFS contrôlés par Pacemaker.

[]

> Pour que plusieurs services BeeGFS du même type soient démarrés sur le même nœud, Pacemaker est configuré pour démarrer les services BeeGFS à l'aide de la méthode de configuration Multi-mode. Pour plus d'informations, reportez-vous à la section "Documentation BeeGFS sur Multi-mode".

Comme les services BeeGFS doivent pouvoir démarrer sur plusieurs nœuds, le fichier de configuration pour chaque service (normalement situé à `/etc/beegfs`) Est stocké sur l'un des volumes E-Series utilisés comme cible BeeGFS pour ce service. Cela rend la configuration et les données d'un service BeeGFS accessibles à tous les nœuds qui peuvent avoir besoin d'exécuter le service.

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

Vérification de la conception

Le design de deuxième génération de la solution BeeGFS sur NetApp a été vérifié à l'aide de trois profils de configuration d'élément de base.

Les profils de configuration incluent les éléments suivants :

- Un élément de base unique, incluant la gestion BeeGFS, les métadonnées et les services de stockage.
- Des métadonnées BeeGFS plus un élément de base de stockage.
- Un élément de base BeeGFS uniquement pour le stockage.

Les éléments de base ont été reliés à deux switchs Mellanox Quantum InfiniBand (MQM8700). Dix clients BeeGFS étaient également connectés aux commutateurs InfiniBand et utilisés pour exécuter des utilitaires de banc d'essai synthétiques.

La figure suivante montre la configuration BeeGFS utilisée pour valider la solution BeeGFS sur NetApp.

[]

Répartition des fichiers BeeGFS

Les systèmes de fichiers parallèles ont notamment pour avantage de répartir les fichiers individuels sur plusieurs cibles de stockage, qui peuvent représenter des volumes sur les mêmes systèmes de stockage sous-jacents ou différents.

Dans BeeGFS, vous pouvez configurer la répartition par répertoire et par fichier pour contrôler le nombre de cibles utilisées pour chaque fichier et pour contrôler la taille chunksize (ou taille de bloc) utilisée pour chaque bande de fichier. Cette configuration permet au système de fichiers de prendre en charge différents types de charges de travail et de profils d'E/S sans avoir à reconfigurer ou à redémarrer des services. Vous pouvez appliquer les paramètres de bande à l'aide du `beegfs-ctl` Outil de ligne de commande ou avec des

applications qui utilisent l'API de répartition. Pour plus d'informations, consultez la documentation BeeGFS pour "Répartition" et "API de répartition".

Pour obtenir les meilleures performances, les motifs de bande ont été ajustés tout au long des tests, et les paramètres utilisés pour chaque test sont notés.

Tests de bande passante IOR : plusieurs clients

Les tests de bande passante IOR ont utilisé OpenMPI pour exécuter des travaux parallèles du générateur d'E/S synthétique IOR (disponible à partir de "GitHub HPC") Sur l'ensemble des 10 nœuds clients à un ou plusieurs blocs de construction BeeGFS. Sauf mention contraire :

- Tous les tests ont utilisé des E/S directes avec une taille de transfert de 1MiB.
- La répartition des fichiers BeeGFS est définie sur une taille chunksize de 1 Mo et une cible par fichier.

Les paramètres suivants ont été utilisés pour IOR avec le nombre de segments ajusté afin de maintenir la taille de fichier d'agrégat à 5 Tio pour un élément de base et 40 Tio pour trois éléments de base.

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

Un élément de base BeeGFS (gestion, métadonnées et stockage)

La figure suivante montre les résultats du test IOR avec un seul élément de base BeeGFS (gestion, métadonnées et stockage).

[]

Métadonnées BeeGFS + élément de base du stockage

La figure suivante présente les résultats du test IOR avec un seul élément de base de stockage + métadonnées BeeGFS.

[]

Élément de base BeeGFS uniquement pour le stockage

La figure suivante montre les résultats du test IOR avec un seul élément de base BeeGFS Storage uniquement.

[]

Trois éléments de base BeeGFS

La figure suivante montre les résultats du test IOR avec trois éléments de base BeeGFS.

[]

Comme on pouvait s'y attendre, la différence de performances entre l'élément de base et les métadonnées suivantes + l'élément de base du stockage est négligeable. En comparant les métadonnées + l'élément de base du stockage et un élément de base uniquement destiné au stockage, on constate une légère augmentation des performances de lecture en raison des disques supplémentaires utilisés comme cibles de stockage. Toutefois, il n'y a pas de différence significative dans les performances d'écriture. Pour améliorer les performances, vous pouvez ajouter plusieurs éléments de base pour faire évoluer les performances de manière linéaire.

Tests de bande passante IOR : client unique

Le test de bande passante IOR a utilisé OpenMPI pour exécuter plusieurs processus IOR à l'aide d'un seul serveur GPU hautes performances afin d'explorer les performances réalisables pour un même client.

Ce test compare également le comportement et les performances de relecture de BeeGFS lorsque le client est configuré pour utiliser le cache de page du noyau Linux (`tuneFileCacheType = native`) par rapport à la valeur par défaut `buffered` réglage.

Le mode de mise en cache native utilise le cache de page du noyau Linux sur le client, ce qui permet aux opérations de relecture de provenir de la mémoire locale au lieu d'être retransmises sur le réseau.

Le diagramme suivant montre les résultats du test IOR avec trois éléments de base BeeGFS et un seul client.

[]

> La répartition BeeGFS pour ces tests a été définie sur une taille chunksize de 1 Mo avec huit cibles par fichier.

Bien que les performances d'écriture et de lecture initiale soient supérieures en mode tampon par défaut, pour les charges de travail qui relisent plusieurs fois les mêmes données, le mode de mise en cache natif a permis d'optimiser considérablement les performances. Cette amélioration des performances de relecture est importante pour les charges de travail telles que l'apprentissage profond qui relire le même dataset plusieurs fois sur plusieurs séries de tests.

Test de performance des métadonnées

Les tests de performance des métadonnées ont utilisé l'outil MDTest (inclus dans IOR) pour mesurer la performance des métadonnées de BeeGFS. Les tests ont utilisé OpenMPI pour exécuter des travaux parallèles sur les dix nœuds clients.

Les paramètres suivants ont été utilisés pour exécuter le test de référence avec le nombre total de processus passe de 10 à 320 par pas de 2x et avec une taille de fichier de 4 ko.

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

Les performances des métadonnées ont été mesurées en premier avec un ou deux blocs de base de stockage + métadonnées afin de montrer l'évolution des performances en ajoutant des éléments de base supplémentaires.

Un seul élément de base de métadonnées BeeGFS + stockage

Le diagramme suivant montre les résultats MDTest avec un bloc de construction BeeGFS + stockage.

[]

Deux métadonnées BeeGFS + éléments de base du stockage

Le diagramme suivant montre les résultats MDTest avec deux métadonnées BeeGFS + des modules de stockage.

[]

Validation fonctionnelle

Dans le cadre de la validation de cette architecture, NetApp a effectué plusieurs tests fonctionnels :

- Défaillance d'un seul port InfiniBand client en désactivant le port de commutateur.
- Défaillance d'un seul port InfiniBand de serveur en désactivant le port du commutateur.
- Déclenchement d'une mise hors tension immédiate d'un serveur à l'aide du contrôleur BMC.
- Placement normal d'un nœud en veille et basculement de service vers un autre nœud.
- Il est normal de remettre un nœud en ligne et de renvoyer les services vers le nœud d'origine.
- Mise hors tension de l'un des commutateurs InfiniBand à l'aide de la PDU. Tous les tests ont été réalisés alors que les tests de stress étaient en cours avec le `sysSessionChecksEnabled: false` Paramètre défini sur les clients BeeGFS. Aucune erreur ni interruption des E/S n'a été observée.

> Il y a un problème connu (voir "Changement") Lorsque les connexions RDMA BeeGFS client/serveur sont interrompues de façon inattendue, soit par la perte de l'interface principale (comme défini dans la section `connInterfacesFile`) Ou un serveur BeeGFS est défaillant ; les E/S du client actif peuvent se bloquer pendant dix minutes avant de reprendre. Ce problème ne se produit pas lorsque les nœuds BeeGFS sont correctement placés en attente pour la maintenance planifiée ou si TCP est utilisé.

Validation du superPOD et du BasePOD NVIDIA DGX A100

NetApp a validé une solution de stockage pour NVIDIA DGX A100 SuperPOD à l'aide d'un système de fichiers BeeGFS constitué de trois éléments de base avec les métadonnées plus le profil de configuration du stockage appliqué. L'effort de qualification a participé au test de la solution décrite par cette architecture NVA avec vingt serveurs GPU DGX A100 exécutant plusieurs bancs d'essai de stockage, d'apprentissage machine et d'apprentissage profond. Tout le stockage certifié pour une utilisation dans le superPOD DGX A100 de NVIDIA est automatiquement certifié pour une utilisation dans les architectures NVIDIA BasePOD.

Pour plus d'informations, voir "NVIDIA DGX SuperPOD avec NetApp" et "NVIDIA DGX BasePOD".

Instructions de dimensionnement

La solution BeeGFS inclut des recommandations sur le dimensionnement de la performance et de la capacité qui étaient basées sur des tests de vérification.

L'objectif de l'architecture modulaire est de créer une solution simple à dimensionner en ajoutant plusieurs éléments de base pour répondre aux exigences d'un système BeeGFS particulier. À l'aide des lignes directrices ci-dessous, vous pouvez estimer la quantité et les types de blocs de construction BeeGFS qui sont nécessaires pour répondre aux exigences de votre environnement.

Notez que ces estimations sont les meilleures performances au cas par cas. Les applications de test des performances synthétiques sont écrites et utilisées pour optimiser l'utilisation des systèmes de fichiers sous-jacents d'une manière qui n'est pas forcément possible pour les applications réelles.

Dimensionnement de la performance

Le tableau suivant indique le dimensionnement de performance recommandé.

Profil de configuration	1MiB lit	Écritures 1MiB
Métadonnées + stockage	62 GiBps	21 GiBps
Stockage uniquement	64 GiBps	21 GiBps

Le dimensionnement de la capacité des métadonnées est basé sur la « règle générale » selon laquelle 500 Go de capacité suffisent pour environ 150 millions de fichiers sur BeeGFS. (Pour plus d'informations, consultez la documentation BeeGFS pour "Configuration minimale requise".)

L'utilisation de fonctions telles que les listes de contrôle d'accès et le nombre de répertoires et de fichiers par répertoire affecte également la vitesse de consommation de l'espace de métadonnées. Les estimations de capacité de stockage tiennent compte de la capacité de disque utilisable ainsi que de la surcharge RAID 6 et XFS.

Dimensionnement de la capacité pour les métadonnées + éléments de base du stockage

Le tableau suivant indique le dimensionnement de la capacité recommandé pour les métadonnées et les éléments de base du stockage.

Taille du disque (2+2 RAID 1) groupes de volumes de métadonnées	Capacité des métadonnées (nombre de fichiers)	Taille des disques (RAID 6 8+2) groupes de volumes de stockage	Capacité de stockage (contenu de fichiers)
1,92 TO	1,938,577,200	1,92 TO	51,77 TO
3,84 TO	3,880,388,400	3,84 TO	103,55 TO
7,68 TO	8,125,278,000	7,68 TO	28.74 TO
15,3 TO	17,269,854,000	15,3 TO	460.60 TO

Lors du dimensionnement des métadonnées et des éléments de base de stockage, vous pouvez réduire les coûts en utilisant des disques plus petits pour les groupes de volumes de métadonnées et les groupes de volumes de stockage.

Dimensionnement de la capacité pour les éléments de base uniquement destinés au stockage

Le tableau suivant indique la règle générale de dimensionnement de la capacité pour les éléments de base uniquement liés au stockage.

Taille des disques (RAID 6 10+2) groupes de volumes de stockage	Capacité de stockage (contenu de fichiers)
1,92 TO	59,89 TO
3,84 TO	11980 TO
7,68 TO	251,89TB
15,3 TO	58,55 TO

Les performances et la surcharge liée à la capacité de l'inclusion du service de gestion dans le premier élément de base sont minimales, sauf si le verrouillage global des fichiers est activé.

Réglage des performances

La solution BeeGFS inclut des recommandations sur le réglage de la performance qui étaient basées sur des tests de vérification.

Bien que BeeGFS fournit des performances raisonnables, NetApp a développé un ensemble de paramètres d'ajustement recommandés pour optimiser les performances. Ces paramètres prennent en compte les fonctionnalités des nœuds de bloc E-Series sous-jacents et les exigences spéciales requises pour exécuter BeeGFS dans une architecture HA à disque partagé.

L'ajustement des performances des nœuds de fichiers

Les paramètres de réglage disponibles que vous pouvez configurer sont les suivants :

1. **Paramètres système dans l'UEFI/BIOS des nœuds de fichiers.** pour optimiser les performances, nous vous recommandons de configurer les paramètres système sur le modèle de serveur que vous utilisez comme nœuds de fichiers. Vous configurez les paramètres système lorsque vous configurez vos nœuds de fichiers à l'aide de la configuration du système (UEFI/BIOS) ou des API Redfish fournies par le contrôleur de gestion de la carte mère (BMC).

   Les paramètres système varient en fonction du modèle de serveur que vous utilisez comme nœud de fichier. Les paramètres doivent être configurés manuellement en fonction du modèle de serveur utilisé. Pour savoir comment configurer les paramètres système des nœuds de fichiers Lenovo SR665 validés, reportez-vous à la section "Réglez les paramètres du système de nœud de fichiers en fonction des performances".

2. **Paramètres par défaut pour les paramètres de configuration requis.** les paramètres de configuration requis affectent la configuration des services BeeGFS et la façon dont les volumes E-Series (dispositifs de bloc) sont formatés et montés par Pacemaker. Voici les paramètres de configuration requis :
   - Paramètres de configuration du service BeeGFS

     Vous pouvez remplacer les paramètres par défaut des paramètres de configuration selon vos besoins. Pour connaître les paramètres que vous pouvez adapter en fonction de vos charges de travail ou de vos cas d'utilisation spécifiques, consultez le "Paramètres de configuration du service BeeGFS".

   - Le formatage de volume et les paramètres de montage sont définis sur les valeurs par défaut recommandées et ne doivent être ajustés que pour des cas d'utilisation avancés. Les valeurs par défaut sont les suivantes :
     - Optimiser le formatage du volume initial en fonction du type de cible (gestion, métadonnées ou stockage, par exemple), de la configuration RAID et de la taille du segment du volume sous-jacent.
     - Réglez la manière dont Pacemaker monte chaque volume pour vous assurer que les modifications sont immédiatement transférées vers les nœuds de blocs E-Series. Cela empêche la perte de données en cas d'échec des nœuds de fichier pour les écritures actives.

       Pour connaître les paramètres que vous pouvez adapter en fonction de vos charges de travail ou de vos cas d'utilisation spécifiques, consultez le "formatage du volume et paramètres de configuration du montage".

3. **Paramètres système dans le système d'exploitation Linux installé sur les nœuds de fichiers.** vous pouvez remplacer les paramètres système d'exploitation Linux par défaut lorsque vous créez l'inventaire Ansible à l'étape 4 de "Créez l'inventaire Ansible".

   Les paramètres par défaut ont été utilisés pour valider la solution BeeGFS sur NetApp, mais vous pouvez

les modifier pour s'adapter à vos workloads ou à vos utilisations spécifiques. Voici quelques exemples de paramètres système d'exploitation Linux que vous pouvez modifier :

- Files d'attente des E/S sur les dispositifs de bloc E-Series.

  Vous pouvez configurer des files d'attente d'E/S sur les périphériques de bloc E-Series utilisés comme cibles BeeGFS pour :

  - Réglez l'algorithme de planification en fonction du type de périphérique (NVMe, HDD, etc.).
  - Augmenter le nombre de demandes en attente.
  - Réglez les tailles des demandes.
  - Optimisez le comportement de lecture anticipée.
- Paramètres de la mémoire virtuelle.

  Vous pouvez régler les paramètres de la mémoire virtuelle pour des performances de diffusion optimales en continu.

- Paramètres CPU.

  Vous pouvez régler le régulateur de fréquence de l'UC et d'autres configurations de l'UC pour obtenir des performances maximales.

- Taille de la demande de lecture.

  Vous pouvez augmenter la taille maximale des demandes de lecture pour les HCA Mellanox.

Réglage des performances des nœuds en mode bloc

En fonction des profils de configuration appliqués à un élément de base BeeGFS particulier, les groupes de volumes configurés sur les nœuds de blocs changent légèrement. Par exemple, avec un nœud de bloc EF600 de 24 disques :

- Pour un seul élément de base, y compris la gestion BeeGFS, les métadonnées et les services de stockage :
  - 1 groupe de volumes RAID 10 2+2 pour la gestion BeeGFS et les services de métadonnées
  - 2 groupes de volumes RAID 6 8+2 pour les services de stockage BeeGFS
- Pour un élément de base de métadonnées + de stockage BeeGFS :
  - 1 groupe de volumes RAID 10 2+2 pour les services de métadonnées BeeGFS
  - 2 groupes de volumes RAID 6 8+2 pour les services de stockage BeeGFS
- Pour l'élément de base de stockage BeeGFS uniquement :
  - 2 groupes de volumes RAID 6 10+2 pour les services de stockage BeeGFS

> Comme BeeGFS a besoin d'un espace de stockage considérable pour la gestion et les métadonnées par rapport au stockage, une seule option consiste à utiliser des disques plus petits pour les groupes de volumes RAID 10. Les lecteurs plus petits doivent être insérés dans les emplacements de lecteur les plus extérieurs. Pour plus d'informations, reportez-vous à la section "instructions de déploiement".

Tous ces paramètres sont configurés par le déploiement Ansible, et plusieurs autres paramètres sont

généralement recommandés pour optimiser les performances/comportements :

- Ajustement de la taille du bloc de cache global à 32Kio et ajustement de la vidage du cache à la demande à 80 %.
- Désactivation de l'équilibrage automatique (en veillant à ce que les attributions de volume du contrôleur restent telles que prévues).
- Activation de la mise en cache de lecture et désactivation de la mise en cache de lecture anticipée
- Activation de la mise en cache d'écriture avec la mise en miroir et demande de sauvegarde sur batterie, les caches sont donc conservés suite à la panne d'un contrôleur de nœud bloc.
- Spécification de l'ordre dans lequel les disques sont affectés aux groupes de volumes, en équilibrant les E/S entre les canaux de disque disponibles.

Élément de base haute capacité

La solution BeeGFS est conçue de façon très performante. Les clients recherchant des cas d'utilisation de grande capacité doivent observer les variations des caractéristiques de conception et de performances décrites ici.

Configuration matérielle et logicielle

Les configurations matérielles et logicielles de l'élément de base haute capacité sont standard, mais les contrôleurs EF600 doivent être remplacés par des contrôleurs EF300, avec une option qui permet de connecter entre 1 et 7 tiroirs d'extension IOM avec 60 disques chacun pour chaque baie de stockage, un total de 2 à 14 tiroirs d'extension par module.

Les clients qui déploient un design d'éléments de base haute capacité n'utilisent probablement que la configuration de type élément de base, qui comprend la gestion BeeGFS, les métadonnées et les services de stockage pour chaque nœud. Pour garantir une rentabilité accrue, les nœuds de stockage haute capacité doivent provisionner des volumes de métadonnées sur les disques NVMe du boîtier de contrôleur EF300 et provisionner les volumes de stockage sur les disques NL-SAS des tiroirs d'extension.

[]

Instructions de dimensionnement

Ces recommandations de dimensionnement supposent que les blocs de base haute capacité sont configurés avec un groupe de volumes SSD NVMe 2+2 pour les métadonnées dans le boîtier EF300 de base et six groupes de volumes NL-SAS 8+2 par plateau d'extension IOM pour le stockage.

Taille du disque (disques durs haute capacité)	Capacité par BB (1 plateau)	Capacité par BB (2 plateaux)	Capacité par BB (3 plateaux)	Capacité par BB (4 plateaux)
4 TO	43TB	878 TO	1317 TO	1756 TO
8 TO	878 TO	1756 TO	2634 TO	3512 TO
10 TO	1097 TO	2195 TO	3292 TO	4390 TO
12 To	1317 TO	2634 TO	3951 TO	5268 TO
16 TO	1756 TO	3512 TO	5268 TO	7024 TO
18 TO	1975 TO	3951 TO	5927 TO	7902 TO

Déploiement de la solution

Présentation du déploiement

Vous pouvez déployer BeeGFS sur NetApp pour des nœuds de fichiers et de blocs validés en utilisant la deuxième génération de l'architecture de base BeeGFS de NetApp.

Collections et rôles Ansible

Vous déployez la solution BeeGFS sur NetApp à l'aide d'Ansible, un moteur d'automatisation IT très populaire qui permet d'automatiser les déploiements d'applications. Ansible utilise ensemble de fichiers appelés « inventaire », qui modélise le système de fichiers BeeGFS que vous souhaitez déployer.

Ansible permet à des entreprises telles que NetApp de développer leurs fonctionnalités intégrées à l'aide des collections sur Ansible Galaxy (voir "Collection NetApp E-Series BeeGFS"). Les collections incluent des modules qui effectuent une fonction ou une tâche spécifique (comme créer un volume E-Series) et incluent des rôles qui peuvent appeler plusieurs modules et d'autres rôles. Cette approche automatisée réduit la durée de déploiement du système de fichiers BeeGFS et du cluster HA sous-jacent. Il simplifie en outre l'ajout d'éléments de base pour développer les systèmes de fichiers existants.

Pour plus de détails, voir "Découvrez l'inventaire Ansible".

> Comme de nombreuses étapes sont nécessaires pour déployer la solution BeeGFS sur NetApp, NetApp ne prend pas en charge le déploiement manuel de la solution.

Profils de configuration pour les éléments de base BeeGFS

Les procédures de déploiement couvrent les profils de configuration suivants :

- Un seul élément de base inclut des services de gestion, de métadonnées et de stockage.
- Un second élément de base qui inclut les métadonnées et les services de stockage.
- Un troisième élément inclut uniquement des services de stockage.

Ces profils illustrent la gamme complète de profils de configuration recommandés pour les éléments de base NetApp BeeGFS. Pour chaque déploiement, le nombre de métadonnées et d'éléments de base de stockage ou d'éléments de base uniquement destinés aux services de stockage peut varier selon les procédures appliquées à la capacité et aux performances.

Présentation des étapes de déploiement

Le déploiement implique plusieurs tâches générales :

Déploiement matériel

1. Assembler physiquement chaque élément de bâtiment.
2. Installez le rack et le matériel de câblage. Pour des procédures détaillées, voir "Déployez le matériel".

De déploiement logiciel

1. "Configurez les nœuds de fichiers et de blocs".
    - Configurez les adresses IP BMC sur les nœuds de fichiers
    - Installez un système d'exploitation pris en charge et configurez la mise en réseau de gestion sur les

nœuds de fichiers

  - Configurez les adresses IP de gestion sur les nœuds de bloc

2. "Configurez un nœud de contrôle Ansible".
3. "Réglez les paramètres du système en fonction des performances".
4. "Créez l'inventaire Ansible".
5. "Définissez l'inventaire Ansible pour les éléments de base BeeGFS".
6. "Déploiement de BeeGFS avec Ansible".
7. "Configurer les clients BeeGFS".

> Les procédures de déploiement incluent plusieurs exemples où du texte doit être copié dans un fichier. Portez une attention particulière aux commentaires en ligne signalés par les caractères "#" ou "//" pour tout ce qui doit ou peut être modifié pour un déploiement spécifique. Par exemple :
>
> ```
> beegfs_ha_ntp_server_pools: # THIS IS AN EXAMPLE OF A COMMENT!
> - "pool 0.pool.ntp.org iburst maxsources 3"
> - "pool 1.pool.ntp.org iburst maxsources 3"
> ```

Architectures dérivées avec variations dans les recommandations de déploiement :

- "Élément de base haute capacité"

Découvrez l'inventaire Ansible

Avant de commencer le déploiement, veillez à bien comprendre comment utiliser Ansible pour configurer et déployer la solution BeeGFS sur NetApp en utilisant la conception d'élément de base BeeGFS deuxième génération.

L'inventaire Ansible définit la configuration des nœuds de fichiers et de blocs et représente le système de fichiers BeeGFS que vous souhaitez déployer. L'inventaire inclut les hôtes, les groupes et les variables décrivant le système de fichiers BeeGFS souhaité. Les inventaires d'échantillons peuvent être téléchargés à partir de "NetApp E-Series BeeGFS GitHub".

Modules et rôles Ansible

Pour appliquer la configuration décrite dans l'inventaire Ansible, utilisez les différents modules et rôles Ansible fournis dans la collection NetApp E-Series Ansible, en particulier le rôle BeeGFS HA 7.2 (disponible sur le "NetApp E-Series BeeGFS GitHub") qui déploie la solution de bout en bout.

Chaque rôle de la collection NetApp E-Series Ansible est un déploiement de bout en bout complet de la solution BeeGFS sur NetApp. Les rôles utilisent les collections NetApp E-Series SANtricity, Host et BeeGFS qui vous permettent de configurer le système de fichiers BeeGFS avec la haute disponibilité. Vous pouvez ensuite provisionner et mapper le stockage, et vérifier que le stockage du cluster est prêt à être utilisé.

Bien que la documentation approfondie soit fournie avec les rôles, les procédures de déploiement décrivent comment utiliser le rôle de déploiement d'une architecture vérifiée NetApp à l'aide de la conception de l'élément de base BeeGFS deuxième génération.

> Bien que la procédure de déploiement tenter d'offrir suffisamment de détails pour que l'expérience précédente avec Ansible ne soit pas une condition préalable, vous devez avoir quelques connaissances de Ansible et de la terminologie connexe.

Disposition de l'inventaire pour un cluster BeeGFS HA

Utilisez la structure d'inventaire Ansible pour définir un cluster BeeGFS HA.

Tout le monde disposant d'une expérience Ansible précédente doit être conscient que le rôle BeeGFS HA implémente une méthode personnalisée pour découvrir quelles variables (ou faits) s'appliquent à chaque hôte. Pour simplifier la création d'un inventaire Ansible, qui décrit les ressources pouvant être exécutées sur plusieurs serveurs.

Un inventaire Ansible comprend généralement les fichiers dans `host_vars` et `group_vars`, et un `inventory.yml` fichier qui affecte des hôtes à des groupes spécifiques (et éventuellement des groupes à d'autres groupes).

> Ne créez pas de fichiers contenant le contenu de cette sous-section, qui est uniquement un exemple.

Bien que cette configuration soit prédéterminée en fonction du profil de configuration, vous devez généralement comprendre comment tout s'établit comme un inventaire Ansible, comme suit :

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
        meta_01:  # Group representing a metadata service with ID 01.
          hosts:
            file_node_01:  # This service is preferred on the first file
node.
            file_node_02:  # And can failover to the second file node.
        meta_02:  # Group representing a metadata service with ID 02.
          hosts:
            file_node_02:  # This service is preferred on the second file
node.
            file_node_01: # And can failover to the first file node.
```

Pour chaque service, un fichier supplémentaire est créé sous `group_vars` description de sa configuration :

```
# meta_01 - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: 8015
  connMetaPortUDP: 8015
  tuneBindToNumaZone: 0
floating_ips:
  - i1b: <IP>/<SUBNET_MASK>
  - i4b: <IP>/<SUBNET_MASK>
# Type of BeeGFS service the HA resource group will manage.
beegfs_service: metadata # Choices: management, metadata, storage.
# What block node should be used to create a volume for this service:
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        owning_controller: A
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.25
```

Cette disposition permet de définir le service, le réseau et la configuration de stockage BeeGFS pour chaque ressource à un seul emplacement. En arrière-plan, le rôle BeeGFS rassemble la configuration nécessaire pour chaque fichier et nœud de bloc en fonction de cette structure d'inventaire. Pour plus d'informations, consultez ce bulletin de blog : "NetApp accélère le déploiement de HA pour BeeGFS avec Ansible".

> L'ID de noeud de type BeeGFS numérique et chaîne pour chaque service est automatiquement configuré en fonction du nom du groupe. Ainsi, en plus de l'exigence générale Ansible pour que les noms de groupe soient uniques, les groupes représentant un service BeeGFS doivent se terminer par un nombre unique pour le type de service BeeGFS que le groupe représente. Par exemple, META_01 et stor_01 sont autorisés, mais Metadata_01 et META_01 ne le sont pas.

Passez en revue les bonnes pratiques

Suivez les bonnes pratiques pour déployer la solution BeeGFS sur NetApp.

Conventions standard

Lors du montage physique et de la création du fichier d'inventaire Ansible, respecter les conventions standard suivantes (pour plus d'informations, voir "Créez l'inventaire Ansible").

- Les noms d'hôte de nœud de fichiers sont numérotés séquentiellement (h01-HN), les chiffres inférieurs se trouvant en haut du rack et les chiffres plus élevés en bas.

  Par exemple, la convention de dénomination `[location][row][rack]hN` ressemble à : `ictad22h01`.

- Chaque nœud de bloc comprend deux contrôleurs de stockage, chacun avec son propre nom d'hôte.

  Un nom de baie de stockage est utilisé pour désigner l'ensemble du système de stockage en mode bloc dans le cadre d'un inventaire Ansible. Les noms de matrice de stockage doivent être numérotés de façon séquentielle (a01 - an), et les noms d'hôte des contrôleurs individuels sont dérivés de cette convention de nommage.

  Par exemple, un nœud de bloc nommé `ictad22a01` en général, des noms d'hôte peuvent être configurés pour chaque contrôleur comme `ictad22a01-a` et `ictad22a01-b`, Mais être mentionné dans un inventaire Ansible comme `ictad22a01`.

- Les nœuds de fichiers et de blocs du même bloc de construction partagent le même schéma de numérotation et sont adjacents les uns aux autres dans le rack, les deux nœuds de fichiers étant situés en haut et les deux nœuds de bloc directement en dessous.

  Par exemple, dans le premier module, les nœuds de fichiers h01 et h02 sont tous deux connectés directement aux nœuds de bloc a01 et a02. De haut en bas, les noms d'hôte sont h01, h02, a01 et a02.

- Les blocs de construction sont installés dans un ordre séquentiel en fonction de leurs noms d'hôtes, de sorte que les noms d'hôtes aux numéros inférieurs se trouvent en haut du rack et les noms d'hôtes aux numéros supérieurs se trouvent en bas.

  L'objectif est de réduire la longueur du câble qui passe en haut des commutateurs du rack et de définir une pratique de déploiement standard afin de simplifier le dépannage. Pour les centres de données où cela n'est pas autorisé en raison de problèmes liés à la stabilité des racks, l'inverse est certainement autorisé, en remplissant le rack par le bas vers le haut.

Configuration du réseau de stockage InfiniBand

Moitié des ports InfiniBand sur chaque nœud de fichiers sont utilisés pour se connecter directement aux nœuds de bloc. L'autre moitié est connectée aux commutateurs InfiniBand et est utilisée pour la connectivité client-serveur BeeGFS. Pour déterminer la taille des sous-réseaux IPoIB utilisés pour les clients et les serveurs BeeGFS, vous devez tenir compte de la croissance prévue de votre cluster Compute/GPU et de votre système de fichiers BeeGFS. Si vous devez vous écarter des plages IP recommandées, n'oubliez pas que chaque connexion directe d'un bloc de construction unique possède un sous-réseau unique et qu'il n'y a pas de chevauchement avec les sous-réseaux utilisés pour la connectivité client-serveur.

Connexions directes

Les nœuds de fichiers et de blocs dans chaque bloc de construction utilisent toujours les adresses IP du tableau suivant pour leurs connexions directes.

> Ce schéma d'adressage adhère à la règle suivante : le troisième octet est toujours impair ou pair, ce qui dépend du fait que le nœud de fichier est impair ou pair.

Nœud de fichier	Port IB	Adresse IP	Nœud de bloc	Port IB	IP physique	IP virtuel
Impair (h1)	i1a	192.168.1.10	Impair (c1)	2a	192.168.1.100	192.168.1.101
Impair (h1)	i2a	192.168.3.10	Impair (c1)	2a	192.168.3.100	192.168.3.101
Impair (h1)	i3a	192.168.5.10	Pair (c2)	2a	192.168.5.100	192.168.5.101
Impair (h1)	i4a	192.168.7.10	Pair (c2)	2a	192.168.7.100	192.168.7.101

Nœud de fichier	Port IB	Adresse IP	Nœud de bloc	Port IB	IP physique	IP virtuel
Pair (h2)	i1a	192.168.2.10	Impair (c1)	2b	192.168.2.100	192.168.2.101
Pair (h2)	i2a	192.168.4.10	Impair (c1)	2b	192.168.4.100	192.168.4.101
Pair (h2)	i3a	192.168.6.10	Pair (c2)	2b	192.168.6.100	192.168.6.101
Pair (h2)	i4a	192.168.8.10	Pair (c2)	2b	192.168.8.100	192.168.8.101

Schéma d'adressage IPoIB BeeGFS client-serveur (deux sous-réseaux)

Pour permettre aux clients BeeGFS d'utiliser deux ports InfiniBand, deux sous-réseaux IPoIB sont requis avec la moitié des services de serveur BeeGFS configurés avec une adresse IP préférée sur chaque sous-réseau pour s'assurer que les clients utilisent deux ports InfiniBand pour maximiser la redondance et le débit possible vers le système de fichiers.

Chaque nœud de fichier exécute plusieurs services BeeGFS Server (gestion, métadonnées ou stockage). Pour permettre à chaque service de basculer de manière indépendante vers l'autre nœud de fichiers, chaque service est configuré avec des adresses IP uniques qui peuvent basculer entre les deux nœuds (parfois appelées interfaces logiques ou LIF).

Bien qu'il ne soit pas obligatoire, ce déploiement suppose que les plages de sous-réseau IPoIB suivantes sont utilisées pour ces connexions et définit un modèle d'adressage standard qui applique les règles suivantes :

- Le second octet est toujours impair ou pair, en fonction du fait que le port InfiniBand du nœud de fichiers est impair ou pair.
- Les adresses IP du cluster BeeGFS sont toujours `xxx. 127.100.yyy` ou `xxx.128.100.yyy`.

> En plus de l'interface utilisée pour la gestion du système d'exploitation intrabande, Corosync peut utiliser des interfaces supplémentaires pour la synchronisation et la battements cardiaques du cluster. Cela permet de s'assurer que la perte d'une interface unique n'entraîne pas l'arrêt complet du cluster.

- Le service BeeGFS Management est toujours à `xxx.yyy.101.0` ou `xxx.yyy.102.0`.
- Les services de métadonnées de BeeGFS sont toujours à `xxx.yyy.101.zzz` ou `xxx.yyy.102.zzz`.
- Les services de stockage BeeGFS sont toujours à `xxx.yyy.103.zzz` ou `xxx.yyy.103.zzz`.
- Adresses dans la plage `100.xxx.1.1` à `100.xxx.99.255` sont réservés aux clients.

Sous-réseau A : 100.127.0.0/16

Le tableau suivant indique la plage pour le sous-réseau A : 100.127.0.0/16.

Objectif	Port InfiniBand	Adresse IP ou plage
Cluster BeeGFS IP	i1b	100.127.100.1 - 100.127.100.255
Gestion BeeGFS	i1b	100.127.101.0
Métadonnées BeeGFS	i1b ou i3b	100.127.101.1 - 100.127.101.255
Stockage BeeGFS	i1b ou i3b	100.127.103.1 - 100.127.103.255
Clients BeeGFS	(varie selon le client)	100.127.1.1 - 100.127.99.255

Sous-réseau B : 100.128.0.0/16

Le tableau suivant indique la plage pour le sous-réseau B : 100.128.0.0/16.

Objectif	Port InfiniBand	Adresse IP ou plage
Cluster BeeGFS IP	i4b	100.128.100.1 - 100.128.100.255
Gestion BeeGFS	i2b	100.128.102.0
Métadonnées BeeGFS	i2b ou i4b	100.128.102.1 - 100.128.102.255
Stockage BeeGFS	i2b ou i4b	100.128.104.1 - 100.128.104.255
Clients BeeGFS	(varie selon le client)	100.128.1.1 - 100.128.99.255

> Toutes les adresses IP comprises dans les plages ci-dessus ne sont pas utilisées dans cette architecture vérifiée NetApp. Ils montrent comment les adresses IP peuvent être pré-allouées pour faciliter l'extension du système de fichiers à l'aide d'un schéma d'adressage IP cohérent. Dans ce schéma, les nœuds de fichiers BeeGFS et les ID de service correspondent au quatrième octet d'une plage bien connue d'adresses IP. Le système de fichiers peut évidemment évoluer au-delà de 255 nœuds ou services si nécessaire.

Déployez le matériel

Chaque module comprend deux nœuds de fichiers x86 validés directement connectés à deux nœuds de bloc à l'aide de câbles InfiniBand HDR (200 Go).

> Chaque élément de base inclut deux nœuds de fichiers BeeGFS, un minimum de deux éléments de base est requis pour établir le quorum dans le cluster de basculement. Bien qu'il soit possible de configurer un cluster à deux nœuds, certaines limites à cette configuration permettent d'empêcher le basculement de se produire dans certains scénarios. Si un cluster à deux nœuds est nécessaire, il est également possible d'intégrer un troisième périphérique comme Tiebreaker, bien qu'il ne soit pas couvert dans cette procédure de déploiement.

Sauf mention contraire, les étapes suivantes sont identiques pour chaque élément de base du cluster, que ce soit pour exécuter des métadonnées et des services de stockage BeeGFS ou des services de stockage uniquement.

Étapes

1. Configurez chaque nœud de fichier BeeGFS avec quatre adaptateurs de canal hôte (HCA) à double port PCIe 4.0 ConnectX-6 en mode InfiniBand et installez-les dans les emplacements PCIe 2, 3, 5 et 6.
2. Configurez chaque nœud de bloc BeeGFS avec une carte d'interface hôte (HIC) à deux ports 200 Go et installez la HIC dans chacun de ses deux contrôleurs de stockage.

   Placez les éléments de base de façon à ce que les deux nœuds de fichier BeeGFS se trouvent au-dessus des nœuds de bloc BeeGFS. La figure suivante montre la configuration matérielle correcte pour le bloc de construction BeeGFS (vue arrière).

   []

   > La configuration de l'alimentation électrique pour les cas d'utilisation en production doit généralement utiliser des blocs d'alimentation redondants.

3. Si nécessaire, installez les lecteurs dans chacun des nœuds de bloc BeeGFS.
   a. Si le module sera utilisé pour exécuter des métadonnées et des services de stockage BeeGFS et des disques plus petits sont utilisés pour les volumes de métadonnées, vérifiez qu'ils sont renseignés dans les emplacements de disque les plus à l'extérieur, comme indiqué dans la figure ci-dessous.
   b. Pour toutes les configurations d'éléments de base, si un boîtier de disque n'est pas plein, assurez-vous qu'un nombre égal de disques est utilisé dans les emplacements 0–11 et 12–23 pour des performances optimales.

      []

4. Pour câbler les nœuds de fichier et de bloc, utilisez des câbles en cuivre à connexion directe 1 m InfiniBand HDR de 200 Go, de sorte qu'ils correspondent à la topologie illustrée dans la figure ci-dessous.

   []

   > Les nœuds répartis entre plusieurs éléments de base ne sont jamais directement connectés. Chaque élément de base doit être considéré comme une unité autonome et toute communication entre les éléments de base se fait par le biais de commutateurs réseau.

5. Utilisez des câbles en cuivre à connexion directe 2 m (ou la longueur appropriée) InfiniBand HDR de 200 Go pour raccorder les ports InfiniBand restants de chaque nœud de fichiers aux commutateurs InfiniBand qui seront utilisés pour le réseau de stockage.

   Si des commutateurs InfiniBand redondants sont utilisés, reliez les ports surlignés en vert clair dans la figure suivante aux différents commutateurs.

   []

6. Au besoin, assembler des éléments de construction supplémentaires en suivant les mêmes directives de câblage.

   > Le nombre total d'éléments de base pouvant être déployés dans un rack unique dépend de l'alimentation et du refroidissement disponibles sur chaque site.

Déployez des logiciels

Configurez les nœuds de fichiers et les nœuds en mode bloc

Si la plupart des tâches de configuration logicielle sont automatisées au moyen des collections Ansible fournies par NetApp, vous devez configurer la mise en réseau sur le contrôleur de gestion de la carte de base (BMC) de chaque serveur et configurer le port de gestion sur chaque contrôleur.

Configurez les nœuds de fichiers

1. Configurez la mise en réseau sur le contrôleur de gestion de la carte mère (BMC) de chaque serveur.

   Pour savoir comment configurer le réseau pour les nœuds de fichiers validés du serveur Lenovo SR665, consultez le "Documentation Lenovo ThinkSystem".

> Un contrôleur de gestion de la carte mère (BMC), parfois appelé processeur de service, est le nom générique de la fonctionnalité de gestion hors bande intégrée dans diverses plates-formes de serveurs qui fournissent un accès à distance même si le système d'exploitation n'est pas installé ou accessible. Les fournisseurs vendent généralement cette fonctionnalité avec leur propre marque. Par exemple, sur le Lenovo SR665, le contrôleur BMC est appelé le contrôleur XClarity (XCC)_ de _Lenovo.

2. Configurez les paramètres du système pour des performances maximales.

   Vous configurez les paramètres système à l'aide de la configuration UEFI (anciennement appelée BIOS) ou en utilisant les API Redfish fournies par de nombreux BMCs. Les paramètres système varient en fonction du modèle de serveur utilisé comme nœud de fichier.

   Pour savoir comment configurer les paramètres système des nœuds de fichiers Lenovo SR665 validés, reportez-vous à la section "Réglez les paramètres du système en fonction des performances".

3. Installez Red Hat 8.4 et configurez le nom d'hôte et le port réseau utilisés pour gérer le système d'exploitation, y compris la connectivité SSH à partir du nœud de contrôle Ansible.

   Ne configurez pas d'adresses IP sur l'un des ports InfiniBand pour le moment.

   > Bien qu'il ne soit pas strictement nécessaire, les sections suivantes présument que les noms d'hôte sont numérotés séquentiellement (comme h1-HN) et font référence aux tâches qui doivent être effectuées sur les hôtes impairs et pairs.

4. Utilisez RedHat Subscription Manager pour enregistrer et abonner le système afin de permettre l'installation des packages requis dans les référentiels Red Hat officiels et de limiter les mises à jour vers la version prise en charge de Red Hat : `subscription-manager release --set=8.4`. Pour obtenir des instructions, reportez-vous à la section "Comment enregistrer et souscrire un système RHEL" et "Comment limiter les mises à jour".
5. Activez le référentiel Red Hat contenant les packages requis pour la haute disponibilité.

   ```
   subscription-manager repo-override --repo=rhel-8-for-x86_64
   -highavailability-rpms --add=enabled:1
   ```

6. Mettez à jour tous les micrologiciels de l'HCA ConnectX-6 avec la version recommandée dans "Exigences technologiques".

   Cette mise à jour peut être effectuée en téléchargeant et en exécutant une version de l'outil mlxup qui regroupe le micrologiciel recommandé. Vous pouvez télécharger cet outil à partir de "Mlxup - Utilitaire de mise à jour et de requête" ("guide de l'utilisateur").

Configurez les nœuds en mode bloc

Configurez les nœuds en mode bloc EF600 en configurant le port de gestion sur chaque contrôleur.

1. Configurez le port de gestion sur chaque contrôleur EF600.

   Pour obtenir des instructions sur la configuration des ports, reportez-vous à la section "Centre de documentation E-Series".

2. Vous pouvez également définir le nom de la matrice de stockage pour chaque système.

   La définition d'un nom peut faciliter la référence à chaque système dans les sections suivantes. Pour obtenir des instructions sur la définition du nom de la matrice, accédez à "Centre de documentation E-Series".

> Bien qu'il ne soit pas strictement nécessaire, les rubriques suivantes présument que les noms des matrices de stockage sont numérotés de façon séquentielle (comme c1 - CN) et font référence aux étapes à suivre sur les systèmes pairs ou impairs.

Configurez un nœud de contrôle Ansible

Pour configurer un nœud de contrôle Ansible, vous devez identifier une machine virtuelle ou physique avec un accès réseau aux ports de gestion de tous les nœuds de fichiers et de blocs qui peuvent être utilisés pour configurer la solution.

Les étapes suivantes ont été testées sur CentOS 8.4. Pour connaître les étapes spécifiques à votre distribution Linux préférée, reportez-vous au "Documentation Ansible".

1. Installer Python 3.9 et s'assurer que la version correcte de `pip` est installé.

```
sudo dnf install python3.9 -y
sudo dnf install python39-pip
sudo dnf install sshpass
```

2. Créez des liens symboliques, en vous assurant que le binaire Python 3.9 est utilisé chaque fois `python3` ou `python` est appelé.

```
sudo ln -sf /usr/bin/python3.9 /usr/bin/python3
sudo ln -sf /usr/bin/python3 /usr/bin/python
```

3. Installez les packages Python requis par les collections NetApp BeeGFS.

```
python3 -m pip install ansible cryptography netaddr
```

> Pour vous assurer que vous installez une version prise en charge d'Ansible et de tous les packages Python requis, consultez le fichier Lisezmoi de la collection BeeGFS. Les versions prises en charge sont également présentées dans le "Exigences techniques".

4. Vérifiez que les versions correctes d'Ansible et de Python sont installées.

```
ansible --version
ansible [core 2.11.6]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-
packages/ansible
  ansible collection location =
/root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.9.2 (default, Mar 10 2021, 17:29:56) [GCC 8.4.1
20200928 (Red Hat 8.4.1-1)]
  jinja version = 3.0.2
  libyaml = True
```

5. Stockez les inventaires Ansible utilisés pour décrire le déploiement BeeGFS dans des systèmes de contrôle source tels que Git ou BitBucket, puis installez Git pour interagir avec ces systèmes.

```
sudo dnf install git -y
```

6. Configurez SSH sans mot de passe. C'est le moyen le plus simple de permettre à Ansible d'accéder aux nœuds de fichiers BeeGFS distants à partir du nœud de contrôle Ansible.
   a. Sur le nœud de contrôle Ansible, si nécessaire, générez une paire de clés publiques à l'aide de `ssh-keygen`
   b. Configurez SSH sans mot de passe sur chacun des nœuds de fichiers à l'aide de `ssh-copy-id <ip_or_hostname>`

      Do **NOT** configurez SSH sans mot de passe sur les nœuds de bloc. Cela n'est ni pris en charge ni obligatoire.

7. Utilisez Ansible Galaxy pour installer la version de la collection BeeGFS répertoriée dans "Exigences techniques".

   Cette installation inclut des dépendances Ansible supplémentaires, telles que le logiciel NetApp SANtricity et les collections d'hôtes.

```
ansible-galaxy collection install netapp_eseries.beegfs:==3.0.1
```

Créez l'inventaire Ansible

Pour définir la configuration des nœuds de fichiers et de blocs, vous créez un inventaire Ansible qui représente le système de fichiers BeeGFS que vous souhaitez déployer. L'inventaire inclut les hôtes, les groupes et les variables décrivant le système de fichiers BeeGFS souhaité.

Étape 1 : définir la configuration de tous les éléments de base

Définissez la configuration qui s'applique à tous les blocs de construction, quel que soit le profil de configuration que vous pouvez appliquer individuellement.

Avant de commencer

- Utilisez un système de contrôle source tel que BitBucket ou Git pour stocker le contenu du répertoire contenant les fichiers d'inventaire et de PlayBook Ansible.
- Créer un `.gitignore` Fichier qui spécifie les fichiers que Git doit ignorer. Cela évite de stocker des fichiers volumineux dans Git.

Étapes

1. Sur votre nœud de contrôle Ansible, identifiez un répertoire à utiliser pour stocker les fichiers d'inventaire et de PlayBook Ansible.

   Sauf indication contraire, tous les fichiers et répertoires créés dans cette étape et les étapes suivantes sont créés par rapport à ce répertoire.

2. Créez les sous-répertoires suivants :

   `host_vars`

   `group_vars`

   `packages`

Étape 2 : définir la configuration des nœuds de fichiers et de blocs individuels

Définissez la configuration qui s'applique aux nœuds de fichiers individuels et aux nœuds d'élément de base individuels.

1. Sous `host_vars/`, Créez un fichier pour chaque noeud de fichier BeeGFS nommé `<HOSTNAME>.yml` Avec le contenu suivant, en portant une attention particulière aux notes concernant le contenu à remplir pour les adresses IP de cluster BeeGFS et les noms d'hôte se terminant par des nombres impairs et impairs.

   Initialement, les noms d'interface de nœud de fichier correspondent à ce qui est répertorié ici (comme ib0 ou ibs1f0). Ces noms personnalisés sont configurés dans Étape 4 : définissez la configuration qui doit s'appliquer à tous les nœuds de fichiers.

```
ansible_host: "<MANAGEMENT_IP>"
eseries_ipoib_interfaces:  # Used to configure BeeGFS cluster IP
addresses.
  - name: i1b
    address: 100.127.100. <NUMBER_FROM_HOSTNAME>/16
  - name: i4b
    address: 100.128.100. <NUMBER_FROM_HOSTNAME>/16
beegfs_ha_cluster_node_ips:
  - <MANAGEMENT_IP>
  - <i1b_BEEGFS_CLUSTER_IP>
  - <i4b_BEEGFS_CLUSTER_IP>
# NVMe over InfiniBand storage communication protocol information
# For odd numbered file nodes (i.e., h01, h03, ..):
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.1.10/24
    configure: true
  - name: i2a
    address: 192.168.3.10/24
    configure: true
  - name: i3a
    address: 192.168.5.10/24
    configure: true
  - name: i4a
    address: 192.168.7.10/24
    configure: true
# For even numbered file nodes (i.e., h02, h04, ..):
# NVMe over InfiniBand storage communication protocol information
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.2.10/24
    configure: true
  - name: i2a
    address: 192.168.4.10/24
    configure: true
  - name: i3a
    address: 192.168.6.10/24
    configure: true
  - name: i4a
    address: 192.168.8.10/24
    configure: true
```

> Si vous avez déjà déployé le cluster BeeGFS, vous devez arrêter le cluster avant d'ajouter ou de modifier des adresses IP configurées de manière statique, y compris les adresses IP et IP du cluster utilisées pour NVMe/IB. Cette modification est nécessaire afin que ces modifications prennent effet correctement et ne perturbent pas les opérations du cluster.

2. Sous `host_vars/`, Créez un fichier pour chaque noeud de bloc BeeGFS nommé `<HOSTNAME>.yml` et remplissez-le avec le contenu suivant.

   Faites particulièrement attention aux remarques concernant le contenu à remplir pour les noms de matrices de stockage se terminant par des nombres pairs ou impairs.

   Pour chaque noeud de bloc, créez un fichier et spécifiez `<MANAGEMENT_IP>` Pour un des deux contrôleurs (généralement Un).

```
eseries_system_name: <STORAGE_ARRAY_NAME>
eseries_system_api_url: https://<MANAGEMENT_IP>:8443/devmgr/v2/
eseries_initiator_protocol: nvme_ib
# For odd numbered block nodes (i.e., a01, a03, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101
    - 192.168.2.101
    - 192.168.1.100
    - 192.168.2.100
  controller_b:
    - 192.168.3.101
    - 192.168.4.101
    - 192.168.3.100
    - 192.168.4.100
# For even numbered block nodes (i.e., a02, a04, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.5.101
    - 192.168.6.101
    - 192.168.5.100
    - 192.168.6.100
  controller_b:
    - 192.168.7.101
    - 192.168.8.101
    - 192.168.7.100
    - 192.168.8.100
```

Étape 3 : définissez une configuration à appliquer à tous les nœuds de fichiers et de blocs

Vous pouvez définir une configuration commune à un groupe d'hôtes sous `group_vars` dans un nom de fichier correspondant au groupe. Cela empêche de répéter une configuration partagée à plusieurs endroits.

Description de la tâche

Les hôtes peuvent se trouver dans plusieurs groupes et au moment de l'exécution, Ansible choisit les variables qui s'appliquent à un hôte donné en fonction de ses règles de priorité de variable. (Pour plus d'informations sur ces règles, consultez la documentation Ansible pour "Utilisation de variables".)

Les affectations hôte-groupe sont définies dans le fichier d'inventaire Ansible réel, créé à la fin de cette procédure.

Étape

Dans Ansible, vous pouvez définir n'importe quelle configuration que vous souhaitez appliquer à tous les hôtes dans un groupe appelé `All`. Créez le fichier `group_vars/all.yml` avec le contenu suivant :

```
ansible_python_interpreter: /usr/bin/python3
beegfs_ha_ntp_server_pools:  # Modify the NTP server addressess if
desired.
  - "pool 0.pool.ntp.org iburst maxsources 3"
  - "pool 1.pool.ntp.org iburst maxsources 3"
```

Étape 4 : définissez la configuration qui doit s'appliquer à tous les nœuds de fichiers

La configuration partagée pour les nœuds de fichiers est définie dans un groupe appelé `ha_cluster`. Les étapes de cette section créent la configuration qui doit être incluse dans le `group_vars/ha_cluster.yml` fichier.

Étapes

1. En haut du fichier, définissez les valeurs par défaut, y compris le mot de passe à utiliser comme `sudo` utilisateur sur les nœuds de fichiers.

   ```
   ### ha_cluster Ansible group inventory file.
   # Place all default/common variables for BeeGFS HA cluster resources
   below.
   ### Cluster node defaults
   ansible_ssh_user: root
   ansible_become_password: <PASSWORD>
   eseries_ipoib_default_hook_templates:
     - 99-multihoming.j2 # This is required when configuring additional
   static IPs (for example cluster IPs) when multiple IB ports are in the
   same IPoIB subnet.
   # If the following options are specified, then Ansible will
   automatically reboot nodes when necessary for changes to take effect:
   eseries_common_allow_host_reboot: true
   eseries_common_reboot_test_command: "systemctl --state=active,exited |
   grep eseries_nvme_ib.service"
   ```

> En particulier pour les environnements de production, ne stockez pas de mots de passe en texte brut. Utilisez plutôt Ansible Vault (voir "Cryptage de contenu avec Ansible Vault") ou le `--ask-become-pass` option lors de l'exécution du manuel de vente. Si le `ansible_ssh_user` est déjà `root`, ensuite, vous pouvez omettre le `ansible_become_password`.

2. Vous pouvez également configurer un nom pour le cluster haute disponibilité (HA) et spécifier un utilisateur pour les communications intra-cluster.

   Si vous modifiez le schéma d'adressage IP privé, vous devez également mettre à jour le schéma par défaut `beegfs_ha_mgmtd_floating_ip`. Ceci doit correspondre à ce que vous configurez plus tard pour le groupe de ressources BeeGFS Management.

   Spécifiez un ou plusieurs e-mails qui doivent recevoir des alertes pour les événements du cluster à l'aide de `beegfs_ha_alert_email_list`.

```
### Cluster information
beegfs_ha_firewall_configure: True
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
# The following variables should be adjusted depending on the desired
configuration:
beegfs_ha_cluster_name: hacluster                  # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster              # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword             # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
beegfs_ha_mgmtd_floating_ip: 100.127.101.0         # BeeGFS management
service IP address.
# Email Alerts Configuration
beegfs_ha_enable_alerts: True
beegfs_ha_alert_email_list: ["email@example.com"]  # E-mail recipient
list for notifications when BeeGFS HA resources change or fail.  Often a
distribution list for the team responsible for managing the cluster.
beegfs_ha_alert_conf_ha_group_options:
      mydomain: “example.com”
# The mydomain parameter specifies the local internet domain name. This
is optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com).
# Adjusting the following parameters is optional:
beegfs_ha_alert_timestamp_format: "%Y-%m-%d %H:%M:%S.%N" #%H:%M:%S.%N
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

> Tout en apparence redondant, `beegfs_ha_mgmtd_floating_ip` Est important lorsque vous faites évoluer le système de fichiers BeeGFS au-delà d’un seul cluster HA. Les clusters HA suivants sont déployés sans service de gestion BeeGFS et point supplémentaires sur le service de gestion fourni par le premier cluster.

3. Configurer un agent d’escrime. (Pour plus de détails, voir "Configurer l’escrime dans un cluster Red Hat haute disponibilité".) Le résultat suivant montre des exemples de configuration d’agents d’escrime courants. Choisissez l’une de ces options.

   Pour cette étape, gardez à l’esprit que :

   - Par défaut, l’escrime est activé, mais vous devez configurer un *agent* d’escrime.

- Le `<HOSTNAME>` spécifié dans le `pcmk_host_map` ou `pcmk_host_list` Doit correspondre au nom d'hôte dans l'inventaire Ansible.
- L'utilisation du cluster BeeGFS sans escrime n'est pas prise en charge, particulièrement en production. Cela permet de s'assurer que les services BeeGFS, y compris les dépendances de ressources comme les périphériques de bloc, basculent en raison d'un problème, il n'y a aucun risque d'accès simultané par plusieurs nœuds qui entraînent une corruption du système de fichiers ou tout autre comportement indésirable ou inattendu. Si l'escrime doit être désactivé, reportez-vous aux notes générales du guide de démarrage et de mise en place du rôle BeeGFS HA `beegfs_ha_cluster_crm_config_options["stonith-enabled"]` à faux dans `ha_cluster.yml`.
- Plusieurs dispositifs d'escrime au niveau des nœuds sont disponibles, et le rôle BeeGFS HA peut configurer n'importe quel agent d'escrime disponible dans le référentiel de package Red Hat HA. Si possible, utilisez un agent d'escrime qui fonctionne via l'alimentation sans coupure (UPS) ou l'unité de distribution de l'alimentation en rack (RPDU), Parce que certains agents d'escrime, tels que le contrôleur de gestion de la carte mère (BMC) ou d'autres dispositifs d'éclairage intégrés au serveur, peuvent ne pas répondre à la demande de clôture dans certains scénarios de panne.

```
### Fencing configuration:
# OPTION 1: To enable fencing using APC Power Distribution Units
(PDUs):
beegfs_ha_fencing_agents:
 fence_apc:
   - ipaddr: <PDU_IP_ADDRESS>
     login: <PDU_USERNAME>
     passwd: <PDU_PASSWORD>
     pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>"
# OPTION 2: To enable fencing using the Redfish APIs provided by the
Lenovo XCC (and other BMCs):
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify "1".
beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
# For details on configuring other fencing agents see
https://access.redhat.com/documentation/en-
us/red_hat_enterprise_linux/8/html/configuring_and_managing_high_avai
lability_clusters/assembly_configuring-fencing-configuring-and-
managing-high-availability-clusters.
```

4. Activez le réglage des performances recommandé dans le système d'exploitation Linux.

   Si de nombreux utilisateurs trouvent les paramètres par défaut des paramètres de performance qui fonctionnent généralement bien, vous pouvez également modifier les paramètres par défaut d'une charge de travail donnée. Ainsi, ces recommandations sont incluses dans le rôle BeeGFS, mais ne sont pas activées par défaut pour s'assurer que les utilisateurs connaissent le réglage appliqué à leur système de fichiers.

   Pour activer le réglage des performances, spécifiez :

```
### Performance Configuration:
beegfs_ha_enable_performance_tuning: True
```

5. (Facultatif) vous pouvez régler les paramètres d'ajustement des performances dans le système d'exploitation Linux selon vos besoins.

Pour obtenir une liste complète des paramètres de réglage disponibles que vous pouvez régler, consultez la section Réglages de la performance par défaut du rôle BeeGFS HA dans "E-Series site GitHub BeeGFS". Les valeurs par défaut peuvent être remplacées pour tous les nœuds du cluster dans ce fichier ou dans le `host_vars` fichier pour un nœud individuel.

6. Pour permettre une connectivité complète de 200 Go/HDR entre des nœuds de bloc et de fichier, utilisez le package Open Subnet Manager (OpenSM) de Mellanox Open Fabrics Enterprise distribution (MLNX_OFED). (La boîte de réception `opensm` package ne prend pas en charge la fonctionnalité de virtualisation nécessaire.) Bien que le déploiement à l'aide d'Ansible soit pris en charge, vous devez d'abord télécharger les packages souhaités vers le nœud de contrôle Ansible utilisé pour exécuter le rôle BeeGFS.

   a. À l'aide de `curl` Ou téléchargez les packages pour la version d'OpenSM répertoriée dans la section exigences technologiques du site web de Mellanox vers le `packages/` répertoire. Par exemple :

```
curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm

curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
```

   b. Remplissez les paramètres suivants dans `group_vars/ha_cluster.yml` (réglez les colis si nécessaire) :

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
eseries_ib_opensm_options:
  virt_enabled: "2"
```

7. Configurer le `udev` Règle pour assurer un mappage cohérent des identificateurs de port InfiniBand logiques aux périphériques PCIe sous-jacents.

   Le `udev` La règle doit être unique à la topologie PCIe de chaque plate-forme de serveur utilisée comme nœud de fichier BeeGFS.

   Utilisez les valeurs suivantes pour les nœuds de fichiers vérifiés :

```
### Ensure Consistent Logical IB Port Numbering
# OPTION 1: Lenovo SR665 PCIe address-to-logical IB port mapping:
eseries_ipoib_udev_rules:
  "0000:41:00.0": i1a
  "0000:41:00.1": i1b
  "0000:01:00.0": i2a
  "0000:01:00.1": i2b
  "0000:a1:00.0": i3a
  "0000:a1:00.1": i3b
  "0000:81:00.0": i4a
  "0000:81:00.1": i4b

# Note: At this time no other x86 servers have been qualified.
Configuration for future qualified file nodes will be added here.
```

8. (Facultatif) mettre à jour l'algorithme de sélection de cible de métadonnées.

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  tuneTargetChooser: randomrobin
```

> Lors des tests de vérification, `randomrobin` Est généralement utilisé pour s'assurer que les fichiers de test étaient répartis de façon égale sur toutes les cibles de stockage BeeGFS pendant l'évaluation des performances (pour plus d'informations sur l'analyse comparative, consultez le site BeeGFS pour "Analyse comparative d'un système BeeGFS"). Avec une utilisation réelle, il est possible que les cibles numérotées soient plus rapidement que les cibles numérotées plus élevées. Omission `randomrobin` et il suffit d'utiliser la valeur par défaut `randomized` la valeur a été indiquée pour fournir de bonnes performances tout en utilisant toujours toutes les cibles disponibles.

Étape 5 : définir la configuration pour le nœud de bloc commun

La configuration partagée pour les nœuds de bloc est définie dans un groupe appelé `eseries_storage_systems`. Les étapes de cette section créent la configuration qui doit être incluse dans le `group_vars/ eseries_storage_systems.yml` fichier.

Étapes

1. Définissez la connexion Ansible sur local, indiquez le mot de passe système et spécifiez si les certificats SSL doivent être vérifiés. (Normalement, Ansible utilise SSH pour la connexion aux hôtes gérés, mais dans le cas des systèmes de stockage NetApp E-Series utilisés comme nœuds de bloc, les modules utilisent l'API REST pour la communication.) En haut du fichier, ajoutez ce qui suit :

```
### eseries_storage_systems Ansible group inventory file.
# Place all default/common variables for NetApp E-Series Storage Systems
here:
ansible_connection: local
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> La liste des mots de passe en texte clair n'est pas recommandée. Utilisez un coffre-fort Ansible ou fournissez le `eseries_system_password` Lors de l'exécution d'Ansible avec `--extra-vars`.

2. Pour assurer des performances optimales, installez les versions répertoriées pour les nœuds de bloc dans "Exigences techniques".

   Téléchargez les fichiers correspondants à partir du "Site de support NetApp". Vous pouvez les mettre à niveau manuellement ou les inclure dans le `packages/` Répertoire du nœud de contrôle Ansible, puis remplissez les paramètres suivants dans `eseries_storage_systems.yml` Pour la mise à niveau avec Ansible :

```
# Firmware, NVSRAM, and Drive Firmware (modify the filenames as needed):
eseries_firmware_firmware: "packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/N6000-872834-D06.dlp"
```

3. Téléchargez et installez la dernière version du micrologiciel de lecteur disponible pour les lecteurs installés dans vos nœuds de bloc à partir du "Site de support NetApp". Vous pouvez les mettre à niveau manuellement ou les inclure dans le `packages/` Répertoire du nœud de contrôle Ansible, puis remplissez les paramètres suivants dans `eseries_storage_systems.yml` Pour la mise à niveau avec Ansible :

```
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
eseries_drive_firmware_upgrade_drives_online: true
```

> Réglage `eseries_drive_firmware_upgrade_drives_online` à `false` Accélère la mise à niveau, mais ne doit pas être effectuée avant le déploiement de BeeGFS. En effet, ce paramètre nécessite l'arrêt de toutes les E/S des disques avant la mise à niveau afin d'éviter les erreurs d'application. Bien que la mise à niveau en ligne du micrologiciel des lecteurs avant la configuration des volumes soit toujours rapide, nous vous recommandons de toujours définir cette valeur sur `true` pour éviter tout problème par la suite.

4. Pour optimiser les performances, effectuez les modifications suivantes de la configuration globale :

```
# Global Configuration Defaults
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required.
```

5. Pour optimiser le provisionnement et le comportement des volumes, spécifiez les paramètres suivants :

```
# Storage Provisioning Defaults
eseries_volume_size_unit: pct
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99:6,
99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

> La valeur spécifiée pour `eseries_storage_pool_usable_drives` Est spécifique aux nœuds de bloc NetApp EF600 et contrôle l'ordre dans lequel les disques sont affectés aux nouveaux groupes de volumes. Cette commande permet de s'assurer que les E/S de chaque groupe sont réparties de manière homogène entre les canaux des disques back-end.

Définissez l'inventaire Ansible pour les éléments de base BeeGFS

Après avoir défini la structure d'inventaire générale Ansible, définissez la configuration de chaque élément de base dans le système de fichiers BeeGFS.

Ces instructions de déploiement montrent comment déployer un système de fichiers composé d'un élément de base, incluant la gestion, les métadonnées et les services de stockage, un deuxième élément de base avec des métadonnées et des services de stockage, et un troisième élément de base uniquement dédié au stockage.

Ces étapes sont destinées à afficher la gamme complète des profils de configuration standard que vous pouvez utiliser pour configurer les éléments de base NetApp BeeGFS de façon à répondre aux exigences du système de fichiers global BeeGFS.

> Dans les sections suivantes et ceci, ajustez selon les besoins pour générer l'inventaire représentant le système de fichiers BeeGFS que vous voulez déployer. Utilisez notamment des noms d'hôte Ansible qui représentent chaque nœud de bloc ou de fichier et le schéma d'adressage IP souhaité pour le réseau de stockage, afin de vous assurer qu'il peut évoluer jusqu'au nombre de nœuds de fichiers et de clients BeeGFS.

Étape 1 : créez le fichier d'inventaire Ansible

Étapes

1. Créer un nouveau `inventory.yml` file, puis insérez les paramètres suivants en remplaçant les hôtes sous `eseries_storage_systems` si nécessaire pour représenter les nœuds en mode bloc dans votre déploiement. Les noms doivent correspondre au nom utilisé pour `host_vars/<FILENAME>.yml`.

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

   Dans les sections suivantes, vous allez créer des groupes Ansible supplémentaires sous `ha_cluster` Qui représentent les services BeeGFS que vous voulez exécuter dans le cluster.

Étape 2 : configurer l'inventaire d'un élément de base de gestion, de métadonnées et de stockage

Le premier élément de base ou du cluster doit inclure le service de gestion BeeGFS ainsi que les services de métadonnées et de stockage :

Étapes

1. Dans `inventory.yml`, remplissez les paramètres suivants sous `ha_cluster: children`:

```
      # ictad22h01/ictad22h02 HA Pair (mgmt/meta/storage building
block):
        mgmt:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_01:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_01:
          hosts:
```

```
            ictad22h01:
            ictad22h02:
        meta_02:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_02:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_03:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_03:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_04:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_04:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_05:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_05:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_06:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_06:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_07:
          hosts:
            ictad22h02:
            ictad22h01:
```

```
        stor_07:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_08:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

2. Créez le fichier `group_vars/mgmt.yml` et inclure les éléments suivants :

```
# mgmt - BeeGFS HA Management Resource Group
# OPTIONAL: Override default BeeGFS management configuration:
# beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
#  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
floating_ips:
  - i1b: 100.127.101.0/16
  - i2b: 100.128.102.0/16
beegfs_service: management
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 1
            owning_controller: A
```

3. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `meta_01` à `meta_08` à l'aide du modèle suivant, puis remplissez les valeurs des espaces réservés pour chaque service faisant référence au tableau ci-dessous :

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET> # Example: i1b:192.168.120.1/16
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 21.25 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

La taille du volume est indiquée sous forme de pourcentage du pool de stockage global (également appelé groupe de volumes). NetApp recommande fortement de laisser une certaine capacité libre dans chaque pool afin d'autoriser le sur-provisionnement SSD (pour plus d'informations, voir "Présentation de la baie NetApp EF600"). Le pool de stockage, `beegfs_m1_m2_m5_m6`, alloue également 1% de la capacité du pool pour le service de gestion. Ainsi, pour les volumes de métadonnées dans le pool de stockage, `beegfs_m1_m2_m5_m6`, Si vous utilisez des disques de 1,92 To ou 3,84 To, définissez cette valeur sur `21.25`; Pour les lecteurs 7,65 To, définissez cette valeur sur `22.25`; Et pour les disques de 15,3 To, définissez cette valeur sur `23.75`.

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_01.yml	8015	i1b:100.127.101.1/16 i2b:100.128.102.1/16	0	ictad22a01	beegfs_m1_m2_m5_m6	A
meta_02.yml	8025	i2b:100.128.102.2/16 i1b:100.127.101.2/16	0	ictad22a01	beegfs_m1_m2_m5_m6	B
meta_03.yml	8035	i3b:100.127.101.3/16 i4b:100.128.102.3/16	1	ictad22a02	beegfs_m3_m4_m7_m8	A

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_04.yml	8045	i4b:100.128.102.4/16 i3b:100.127.101.4/16	1	ictad22a02	beegfs_m3_m4_m7_m8	B
meta_05.yml	8055	i1b:100.127.101.5/16 i2b:100.128.102.5/16	0	ictad22a01	beegfs_m1_m2_m5_m6	A
meta_06.yml	8065	i2b:100.128.102.6/16 i1b:100.127.101.6/16	0	ictad22a01	beegfs_m1_m2_m5_m6	B
meta_07.yml	8075	i3b:100.127.101.7/16 i4b:100.128.102.7/16	1	ictad22a02	beegfs_m3_m4_m7_m8	A
meta_08.yml	8085	i4b:100.128.102.8/16 i3b:100.127.101.8/16	1	ictad22a02	beegfs_m3_m4_m7_m8	B

4. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `stor_01` à `stor_08` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# stor_0X - BeeGFS HA Storage Resource
Groupbeegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!             owning_controller:
<OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage".

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_01.yml	8013	i1b:100.127.103.1/16 i2b:100.128.104.1/16	0	ictad22a01	beegfs_s1_s2	A
stor_02.yml	8023	i2b:100.128.104.2/16 i1b:100.127.103.2/16	0	ictad22a01	beegfs_s1_s2	B
stor_03.yml	8033	i3b:100.127.103.3/16 i4b:100.128.104.3/16	1	ictad22a02	beegfs_s3_s4	A
stor_04.yml	8043	i4b:100.128.104.4/16 i3b:100.127.103.4/16	1	ictad22a02	beegfs_s3_s4	B

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_05.yml	8053	i1b:100.127.103.5/16 i2b:100.128.104.5/16	0	ictad22a01	beegfs_s5_s6	A
stor_06.yml	8063	i2b:100.128.104.6/16 i1b:100.127.103.6/16	0	ictad22a01	beegfs_s5_s6	B
stor_07.yml	8073	i3b:100.127.103.7/16 i4b:100.128.104.7/16	1	ictad22a02	beegfs_s7_s8	A
stor_08.yml	8083	i4b:100.128.104.8/16 i3b:100.127.103.8/16	1	ictad22a02	beegfs_s7_s8	B

Étape 3 : configurer l'inventaire d'un élément de base métadonnées + stockage

Elles expliquent comment configurer un inventaire Ansible pour un élément de base de stockage + de métadonnées BeeGFS.

Étapes

1. Dans `inventory.yml`, remplissez les paramètres suivants sous la configuration existante :

```
        meta_09:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_09:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_10:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_10:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_11:
          hosts:
            ictad22h03:
```

```
            ictad22h04:
        stor_11:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_12:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_12:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_13:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_13:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_14:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_14:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_15:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_15:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_16:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_16:
          hosts:
            ictad22h04:
            ictad22h03:
```

2. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `meta_09` à `meta_16` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.5 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage".

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_09.yml	8015	i1b:100.127.101.9/16 i2b:100.128.102.9/16	0	ictad22a03	beegfs_m9_m10_m13_m14	A
meta_10.yml	8025	i2b:100.128.102.10/16 i1b:100.127.101.10/16	0	ictad22a03	beegfs_m9_m10_m13_m14	B
meta_11.yml	8035	i3b:100.127.101.11/16 i4b:100.128.102.11/16	1	ictad22a04	beegfs_m11_m12_m15_m16	A
meta_12.yml	8045	i4b:100.128.102.12/16 i3b:100.127.101.12/16	1	ictad22a04	beegfs_m11_m12_m15_m16	B

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_13.yml	8055	i1b:100.127.101.13/16 i2b:100.128.102.13/16	0	ictad22a03	beegfs_m9_m10_m13_m14	A
meta_14.yml	8065	i2b:100.128.102.14/16 i1b:100.127.101.14/16	0	ictad22a03	beegfs_m9_m10_m13_m14	B
meta_15.yml	8075	i3b:100.127.101.15/16 i4b:100.128.102.15/16	1	ictad22a04	beegfs_m11_m12_m15_m16	A
meta_16.yml	8085	i4b:100.128.102.16/16 i3b:100.127.101.16/16	1	ictad22a04	beegfs_m11_m12_m15_m16	B

3. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `stor_09` à `stor_16` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage"..

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_09.yml	8013	i1b:100.127.103.9/16 i2b:100.128.104.9/16	0	ictad22a03	beegfs_s9_s10	A
stor_10.yml	8023	i2b:100.128.104.10/16 i1b:100.127.103.10/16	0	ictad22a03	beegfs_s9_s10	B
stor_11.yml	8033	i3b:100.127.103.11/16 i4b:100.128.104.11/16	1	ictad22a04	beegfs_s11_s12	A
stor_12.yml	8043	i4b:100.128.104.12/16 i3b:100.127.103.12/16	1	ictad22a04	beegfs_s11_s12	B
stor_13.yml	8053	i1b:100.127.103.13/16 i2b:100.128.104.13/16	0	ictad22a03	beegfs_s13_s14	A
stor_14.yml	8063	i2b:100.128.104.14/16 i1b:100.127.103.14/16	0	ictad22a03	beegfs_s13_s14	B
stor_15.yml	8073	i3b:100.127.103.15/16 i4b:100.128.104.15/16	1	ictad22a04	beegfs_s15_s16	A
stor_16.yml	8083	i4b:100.128.104.16/16 i3b:100.127.103.16/16	1	ictad22a04	beegfs_s15_s16	B

Étape 4 : configurer l'inventaire pour un élément de base stockage uniquement

Procédure de configuration d'un inventaire Ansible pour un élément de base BeeGFS Storage uniquement. La différence majeure entre l'installation de la configuration pour un bloc de métadonnées + stockage et un bloc modulaire uniquement destiné au stockage, c'est l'omission de tous les groupes de ressources de métadonnées et la modification `criteria_drive_count` de 10 à 12 pour chaque pool de stockage.

Étapes

1. Dans `inventory.yml`, remplissez les paramètres suivants sous la configuration existante :

```
# ictad22h05/ictad22h06 HA Pair (storage only building block):
  stor_17:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_18:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_19:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_20:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_21:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_22:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_23:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_24:
    hosts:
      ictad22h06:
      ictad22h05:
```

2. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `stor_17` à `stor_24` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 12
        common_volume_configuration:
          segment_size_kb: 512
        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50
            owning_controller: <OWNING CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage".

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_17.yml	8013	i1b:100.127.103.17/16 i2b:100.128.104.17/16	0	ictad22a05	beegfs_s17_s18	A
stor_18.yml	8023	i2b:100.128.104.18/16 i1b:100.127.103.18/16	0	ictad22a05	beegfs_s17_s18	B
stor_19.yml	8033	i3b:100.127.103.19/16 i4b:100.128.104.19/16	1	ictad22a06	beegfs_s19_s20	A
stor_20.yml	8043	i4b:100.128.104.20/16 i3b:100.127.103.20/16	1	ictad22a06	beegfs_s19_s20	B

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_21.yml	8053	i1b:100.127.103.21/16 i2b:100.128.104.21/16	0	ictad22a05	beegfs_s21_s22	A
stor_22.yml	8063	i2b:100.128.104.22/16 i1b:100.127.103.22/16	0	ictad22a05	beegfs_s21_s22	B
stor_23.yml	8073	i3b:100.127.103.23/16 i4b:100.128.104.23/16	1	ictad22a06	beegfs_s23_s24	A
stor_24.yml	8083	i4b:100.128.104.24/16 i3b:100.127.103.24/16	1	ictad22a06	beegfs_s23_s24	B

Déployez BeeGFS

Un déploiement et une gestion de la configuration impliquent d'exécuter un ou plusieurs playbooks contenant les tâches Ansible requises pour exécuter et placer le système global dans l'état souhaité.

Même si toutes les tâches peuvent être incluses dans un seul manuel de vente, il est difficile pour les systèmes complexes de gérer cette tâche très rapidement. Ansible vous permet de créer et de distribuer des rôles comme un moyen de packaging des playbooks réutilisables et du contenu associé (par exemple, variables par défaut, tâches et gestionnaires). Pour plus d'informations, consultez la documentation Ansible pour "Rôles".

Les rôles sont souvent distribués dans le cadre d'une collection Ansible contenant des rôles et des modules associés. Donc, ces playbooks importent principalement plusieurs rôles distribués dans les différentes collections NetApp E-Series Ansible.

> Actuellement, au moins deux éléments de base (quatre nœuds de fichiers) sont nécessaires pour déployer BeeGFS, à moins qu'un périphérique quorum distinct soit configuré comme un disjoncteur d'attache pour limiter les problèmes lors de l'établissement du quorum avec un cluster à deux nœuds.

Étapes

1. Créer un nouveau `playbook.yml` classez et incluez les éléments suivants :

```
# BeeGFS HA (High Availability) cluster playbook.
- hosts: eseries_storage_systems
  gather_facts: false
  collections:
    - netapp_eseries.santricity
```

```
  tasks:
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version
        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'
        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true
        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
groups[beegfs_ha_ansible_storage_group]
    - name: Verify any provided tags are supported.
      fail:
```

```
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_2
```

> Ce PlayBook s'exécute `pre_tasks` Vérifiez que Python 3 est installé sur les nœuds de fichiers et vérifiez que les balises Ansible fournies sont prises en charge.

2. Utilisez le `ansible-playbook` Commande avec les fichiers d'inventaire et de PlayBook lorsque vous êtes prêt à déployer BeeGFS.

   Le déploiement va s'exécuter tout `pre_tasks`, Puis demander confirmation de l'utilisateur avant de poursuivre le déploiement BeeGFS.

   Exécuter la commande suivante en réglant le nombre de fourches selon les besoins (voir la remarque ci-dessous) :

```
ansible-playbook -i inventory.yml playbook.yml --forks 20
```

> En particulier pour les déploiements plus importants, en remplaçant le nombre par défaut de fourches (5) à l'aide du `forks` Paramètre recommandé pour augmenter le nombre d'hôtes qu'Ansible configure en parallèle. (Pour plus d'informations, voir "Réglage des performances Ansible" et "Contrôle de l'exécution de PlayBook".) Le paramètre de valeur maximale dépend de la puissance de traitement disponible sur le nœud de contrôle Ansible. L'exemple ci-dessus de 20 a été exécuté sur un nœud de contrôle Ansible virtuel avec 4 processeurs (Intel® Xeon® Gold 6146 CPU à 3,20 GHz).

   Selon la taille du déploiement et les performances réseau entre le nœud de contrôle Ansible et les nœuds de fichier et bloc BeeGFS, la durée de déploiement peut varier.

Configurer les clients BeeGFS

Vous devez installer et configurer le client BeeGFS sur tous les hôtes qui doivent accéder au système de fichiers BeeGFS, comme les nœuds de calcul ou les nœuds GPU. Pour

cette tâche, vous pouvez utiliser Ansible et la collection BeeGFS.

Étapes

1. Si nécessaire, configurez une connexion SSH sans mot de passe depuis le nœud de contrôle Ansible vers chacun des hôtes que vous souhaitez configurer comme clients BeeGFS :

   ```
   ssh-copy-id <user>@<HOSTNAME_OR_IP>
   ```

2. Sous `host_vars/`, Créez un fichier pour chaque client BeeGFS nommé `<HOSTNAME>.yml` avec le contenu suivant, en renseignant le texte de l'espace réservé contenant les informations correctes pour votre environnement :

   ```
   # BeeGFS Client
   ansible_host: <MANAGEMENT_IP>
   # OPTIONAL: If you want to use the NetApp E-Series Host Collection's
   IPoIB role to configure InfiniBand interfaces for clients to connect to
   BeeGFS file systems:
   eseries_ipoib_interfaces:
     - name: <INTERFACE>
       address: <IP>/<SUBNET_MASK> # Example: 100.127.1. 1/16
     - name: <INTERFACE>0
       address: <IP>/<SUBNET_MASK>
   ```

   Actuellement, deux interfaces InfiniBand doivent être configurées sur chaque client, une dans chacun des deux sous-réseaux de stockage IPoIB. Si vous utilisez les exemples de sous-réseaux et les plages recommandées pour chaque service BeeGFS répertorié ici, les clients doivent avoir une interface configurée dans la plage de `100.127.1. 0` à `100.127.99.255` et l'autre dans `100.128.1. 0` à `100.128. 99.255`.

3. Créez un nouveau fichier `client_inventory.yml`, puis remplissez les paramètres suivants en haut :

   ```
   # BeeGFS client inventory.
   all:
     vars:
       ansible_ssh_user: <USER> # This is the user Ansible should use to
   connect to each client.
       ansible_become_password: <PASSWORD> # This is the password Ansible
   will use for privilege escalation, and requires the ansible_ssh_user be
   root, or have sudo privileges.
   The defaults set by the BeeGFS HA role are based on the testing
   performed as part of this NetApp Verified Architecture and differ from
   the typical BeeGFS client defaults.
   ```

Ne stockez pas les mots de passe en texte brut. Utilisez plutôt Ansible Vault (consultez la documentation Ansible pour "Cryptage de contenu avec Ansible Vault") ou utilisez l' `--ask -become-pass` option lors de l'exécution du manuel de vente.

4. Dans le `client_inventory.yml` Fichier, répertorie tous les hôtes qui doivent être configurés comme clients BeeGFS sous `beegfs_clients` Définissez ensuite toute configuration supplémentaire requise pour générer le module de noyau client BeeGFS.

```
  children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        ictad21h01:
        ictad21h02:
        ictad21h03:
        ictad21h04:
        ictad21h05:
        ictad21h06:
        ictad21h07:
        ictad21h08:
        ictad21h09:
        ictad21h10:
      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        beegfs_client_ofed_enable: True
        beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"
        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        eseries_ib_skip: False # Default value.
        beegfs_client_ofed_enable: False # Default value.
```

Lorsque vous utilisez les pilotes OFED Mellanox, assurez-vous que `beegfs_client_ofed_include_path` Indique le bon « chemin d'accès à l'en-tête » pour votre installation Linux. Pour plus d'informations, consultez la documentation BeeGFS pour "Prise en charge de RDMA".

5. Dans le `client_inventory.yml` Fichier, répertorie les systèmes de fichiers BeeGFS que vous souhaitez monter au bas de tout ce qui a été défini précédemment `vars`.

```
        beegfs_client_mounts:
          - sysMgmtdHost: 100.127.101.0 # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default "sync" mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
```

> Le `beegfs_client_config` représente les paramètres testés. Reportez-vous à la documentation fournie avec le `netapp_eseries.beegfs` collection `beegfs_client` rôle pour une vue d'ensemble complète de toutes les options. Cela inclut le montage de plusieurs systèmes de fichiers BeeGFS ou le montage du même système de fichiers BeeGFS plusieurs fois.

6. Créer un nouveau `client_playbook.yml` puis remplissez les paramètres suivants :

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

> Ignorer l'importation du `netapp_eseries.host` collecte et `ipoib` Rôle si vous avez déjà installé les pilotes IB/RDMA requis et configuré les adresses IP sur les interfaces IPoIB appropriées.

7. Pour installer et construire le client et monter BeeGFS, exécutez la commande suivante :

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

8. Avant de placer le système de fichiers BeeGFS en production, nous vous recommandons **fortement** de vous connecter à n'importe quel client et de l'exécuter `beegfs-fsck --checkfs` afin de garantir que tous les nœuds sont accessibles et qu'aucun problème n'est signalé.

Faites évoluer votre infrastructure au-delà de cinq éléments de base

Vous pouvez configurer Pacemaker et Corosync pour qu'elle dépasse cinq éléments de base (10 nœuds de fichiers). Toutefois, il y a des inconvénients pour les grands clusters, et finalement Pacemaker et Corosync imposent un maximum de 32 nœuds.

NetApp n'a testé que les clusters BeeGFS HA pour jusqu'à 10 nœuds. Il n'est pas recommandé ou pris en charge de faire évoluer des clusters individuels au-delà de cette limite. Toutefois, les systèmes de fichiers BeeGFS nécessitent une évolutivité bien supérieure à 10 nœuds, et NetApp en est responsable dans la solution BeeGFS sur NetApp.

En déployant plusieurs clusters HA contenant un sous-ensemble des éléments de base de chaque système de fichiers, vous pouvez faire évoluer le système de fichiers BeeGFS indépendamment de toutes les limites recommandées ou strictes sur les mécanismes de mise en cluster HA sous-jacents. Dans ce scénario, procédez comme suit :

- Créez un nouvel inventaire Ansible représentant les clusters HA supplémentaires, puis ignorez la configuration d'un autre service de gestion. Pointez plutôt le `beegfs_ha_mgmtd_floating_ip` variable dans chaque cluster supplémentaire `ha_cluster.yml` Vers l'IP pour le premier service de gestion BeeGFS.

- Lorsque vous ajoutez des clusters haute disponibilité supplémentaires sur le même système de fichiers, vérifiez ce qui suit :
  - Les ID de nœud BeeGFS sont uniques.
  - Les noms de fichiers correspondant à chaque service sous `group_vars` est unique dans tous les clusters.
  - Les adresses IP du client et du serveur BeeGFS sont uniques dans tous les clusters.
  - Le premier cluster HA contenant le service de gestion BeeGFS est exécuté avant de tenter de déployer ou de mettre à jour des clusters supplémentaires.
- Maintenir les inventaires pour chaque cluster HA séparément dans leur propre arborescence de répertoires.

  La combinaison des fichiers d'inventaire de plusieurs clusters dans une arborescence de répertoires peut entraîner des problèmes avec la façon dont le rôle BeeGFS HA agrège la configuration appliquée à un cluster donné.

> Chaque cluster haute disponibilité n'a aucune exigence à évoluer jusqu'à cinq éléments de base avant d'en créer un nouveau. Dans bien des cas, la gestion requiert moins d'éléments de base par cluster est plus simple. Une approche consiste à configurer les éléments de base de chaque rack en tant que cluster haute disponibilité.

Pourcentages de surprovisionnement recommandés pour le pool de stockage

Lorsque vous suivez la configuration standard des quatre volumes par pool de stockage pour les éléments de base de deuxième génération, consultez le tableau suivant.

Ce tableau fournit les pourcentages recommandés à utiliser comme taille de volume dans `eseries_storage_pool_configuration` Pour chaque cible de stockage ou de métadonnées BeeGFS :

Taille du disque	Taille
1,92 TO	18
3,84 TO	21.5
7,68 TO	22.5
15,3 TO	24

> Les recommandations ci-dessus ne s'appliquent pas au pool de stockage contenant le service de gestion, ce qui devrait réduire la taille ci-dessus de .25 % pour allouer 1 % du pool de stockage aux données de gestion.

Pour comprendre comment ces valeurs ont été déterminées, reportez-vous à la section "Tr-4800 : Annexe A : compréhension de la longévité et du sur-provisionnement des disques SSD".

Élément de base haute capacité

Le guide de déploiement de la solution BeeGFS standard décrit les procédures et les recommandations pour répondre aux exigences des workloads de haute performance. Les clients cherchant à répondre aux besoins en capacité élevés doivent observer les

variations du déploiement et les recommandations décrites ici.

[]

Contrôleurs

Pour les éléments de base haute capacité, les contrôleurs EF600 doivent être remplacés par des contrôleurs EF300, chacun avec une HIC Cascade installée pour l'extension SAS. Chaque nœud de bloc aura un nombre minimal de SSD NVMe dans le boîtier de la baie pour le stockage de métadonnées BeeGFS et sera relié à des tiroirs d'extension dotés de disques durs NL-SAS pour les volumes de stockage BeeGFS.

La configuration du noeud fichier à noeud bloc reste la même.

Placement des disques

Chaque nœud de bloc exige un minimum de 4 SSD NVMe pour le stockage de métadonnées BeeGFS. Ces lecteurs doivent être placés dans les emplacements les plus extérieurs du boîtier.

[]

Bacs d'extension

L'élément de base haute capacité peut être dimensionné avec 1-7, 60 tiroirs d'extension de disque par matrice de stockage.

Pour obtenir des instructions sur le câble de chaque bac d'extension, "Consultez la section câblage EF300 pour les tiroirs disques".

Informations sur le copyright

Copyright © 2024 NetApp, Inc. Tous droits réservés. Imprimé aux États-Unis. Aucune partie de ce document protégé par copyright ne peut être reproduite sous quelque forme que ce soit ou selon quelque méthode que ce soit (graphique, électronique ou mécanique, notamment par photocopie, enregistrement ou stockage dans un système de récupération électronique) sans l'autorisation écrite préalable du détenteur du droit de copyright.

Les logiciels dérivés des éléments NetApp protégés par copyright sont soumis à la licence et à l'avis de non-responsabilité suivants :

CE LOGICIEL EST FOURNI PAR NETAPP « EN L'ÉTAT » ET SANS GARANTIES EXPRESSES OU TACITES, Y COMPRIS LES GARANTIES TACITES DE QUALITÉ MARCHANDE ET D'ADÉQUATION À UN USAGE PARTICULIER, QUI SONT EXCLUES PAR LES PRÉSENTES. EN AUCUN CAS NETAPP NE SERA TENU POUR RESPONSABLE DE DOMMAGES DIRECTS, INDIRECTS, ACCESSOIRES, PARTICULIERS OU EXEMPLAIRES (Y COMPRIS L'ACHAT DE BIENS ET DE SERVICES DE SUBSTITUTION, LA PERTE DE JOUISSANCE, DE DONNÉES OU DE PROFITS, OU L'INTERRUPTION D'ACTIVITÉ), QUELLES QU'EN SOIENT LA CAUSE ET LA DOCTRINE DE RESPONSABILITÉ, QU'IL S'AGISSE DE RESPONSABILITÉ CONTRACTUELLE, STRICTE OU DÉLICTUELLE (Y COMPRIS LA NÉGLIGENCE OU AUTRE) DÉCOULANT DE L'UTILISATION DE CE LOGICIEL, MÊME SI LA SOCIÉTÉ A ÉTÉ INFORMÉE DE LA POSSIBILITÉ DE TELS DOMMAGES.

NetApp se réserve le droit de modifier les produits décrits dans le présent document à tout moment et sans préavis. NetApp décline toute responsabilité découlant de l'utilisation des produits décrits dans le présent document, sauf accord explicite écrit de NetApp. L'utilisation ou l'achat de ce produit ne concède pas de licence dans le cadre de droits de brevet, de droits de marque commerciale ou de tout autre droit de propriété intellectuelle de NetApp.

Le produit décrit dans ce manuel peut être protégé par un ou plusieurs brevets américains, étrangers ou par une demande en attente.

LÉGENDE DE RESTRICTION DES DROITS : L'utilisation, la duplication ou la divulgation par le gouvernement sont sujettes aux restrictions énoncées dans le sous-paragraphe (b)(3) de la clause Rights in Technical Data-Noncommercial Items du DFARS 252.227-7013 (février 2014) et du FAR 52.227-19 (décembre 2007).

Les données contenues dans les présentes se rapportent à un produit et/ou service commercial (tel que défini par la clause FAR 2.101). Il s'agit de données propriétaires de NetApp, Inc. Toutes les données techniques et tous les logiciels fournis par NetApp en vertu du présent Accord sont à caractère commercial et ont été exclusivement développés à l'aide de fonds privés. Le gouvernement des États-Unis dispose d'une licence limitée irrévocable, non exclusive, non cessible, non transférable et mondiale. Cette licence lui permet d'utiliser uniquement les données relatives au contrat du gouvernement des États-Unis d'après lequel les données lui ont été fournies ou celles qui sont nécessaires à son exécution. Sauf dispositions contraires énoncées dans les présentes, l'utilisation, la divulgation, la reproduction, la modification, l'exécution, l'affichage des données sont interdits sans avoir obtenu le consentement écrit préalable de NetApp, Inc. Les droits de licences du Département de la Défense du gouvernement des États-Unis se limitent aux droits identifiés par la clause 252.227-7015(b) du DFARS (février 2014).

Informations sur les marques commerciales

NETAPP, le logo NETAPP et les marques citées sur le site http://www.netapp.com/TM sont des marques déposées ou des marques commerciales de NetApp, Inc. Les autres noms de marques et de produits sont des marques commerciales de leurs propriétaires respectifs.